CertiControl-NN: Complete Technical Report

Theory, algorithms, implementation and experiments, explained from first principles

CertiControl-NN project

September 2026

Contents

About this report **5**

Part I — Background **6**

1 Motivation: from testing to proof **7**
1.1 The deployment problem . . . 7
1.2 A concrete demonstration from this project . . . 7
1.3 What the project builds . . . 7
1.4 Summary of results . . . 8

2 Mathematical prerequisites **9**
2.1 Vectors, norms and boxes . . . 9
2.2 Minimising a linear function over a box . . . 9
2.3 Derivatives, gradients and Jacobians . . . 9
2.4 The mean value theorem and Lipschitz continuity . . . 10
2.5 Convexity and concavity . . . 10
2.6 Sets: interior, boundary, compactness . . . 10
2.7 Linear programming (LP) in one paragraph . . . 10
2.8 Floating-point arithmetic . . . 10

3 Dynamical systems and control **11**
3.1 States and ordinary differential equations . . . 11
3.2 Controlled systems and the control-affine form . . . 11
3.3 System 1: the torque-limited inverted pendulum . . . 11
3.3.1 Derivation from Newton’s law . . . 11
3.3.2 Parameters and sets . . . 12
3.3.3 Code . . . 12
3.4 System 2: the Dubins car with acceleration . . . 12
3.4.1 Kinematics . . . 12
3.4.2 The periodic heading . . . 13
3.4.3 Sets written as polynomials . . . 13
3.5 Feedback, closed loop and simulation . . . 14

4 Neural networks and differentiation **15**
4.1 The multilayer perceptron . . . 15
4.2 Why tanh? . . . 15
4.3 The controller: saturation by construction . . . 15
4.4 Automatic differentiation . . . 16
4.4.1 Reverse mode . . . 16
4.4.2 Forward mode and dual numbers . . . 16
4.4.3 Why forward mode fits this project exactly . . . 16
4.4.4 Derivation of the tangent recursion . . . 16
4.4.5 Worked example . . . 16
4.4.6 Code . . . 17

4.5 One computation, several meanings: `src/ops.py` . . . 17

Part II — Safety theory **18**

5 Invariance, barrier certificates and the main theorem **19**
5.1 Safety as "never enter the unsafe set" . . . 19
5.2 Forward invariance . . . 19
5.3 Describing sets by functions . . . 19
5.4 Nagumo's theorem . . . 19
5.5 Barrier certificates and control barrier functions . . . 20
5.6 The comparison lemma, with proof . . . 21
5.7 The four certified properties . . . 21
5.8 Main theorem, with proof . . . 21
5.9 Connection to Lyapunov functions . . . 22
5.10 Why *learn* B and π? . . . 22

Part III — Verification **23**

6 Verification fundamentals **24**
6.1 The verification problem . . . 24
6.2 Soundness and completeness . . . 24
6.3 Over-approximation: the central idea . . . 24
6.4 Falsification is easy; proof is hard . . . 25

7 Interval arithmetic and interval bound propagation (IBP) **26**
7.1 Intuition . . . 26
7.2 Definitions and the fundamental theorem . . . 26
7.3 The rules, with justification . . . 26
7.3.1 The affine layer in detail . . . 26
7.3.2 Sine and cosine . . . 27
7.3.3 The derivative of tanh . . . 27
7.4 The dependency problem . . . 27
7.4.1 Worked example: IBP on a tiny network . . . 27
7.5 Floating-point soundness in the implementation . . . 28
7.6 The `Interval` class . . . 28
7.7 Testing soundness . . . 29
7.8 Why IBP, and why it is not enough . . . 29

8 Linear relaxations, CROWN and auto_LiRPA **30**
8.1 Intuition . . . 30
8.2 Linear relaxation of a single nonlinearity . . . 30
8.3 Products: the McCormick envelope . . . 30
8.4 CROWN: backward propagation of linear bounds . . . 31
8.4.1 Worked example (same tiny network) . . . 31
8.5 auto_LiRPA in the project . . . 31
8.5.1 Engineering details and pitfalls . . . 32
8.6 Why CROWN (and not α-CROWN or an LP)? . . . 32

9 SAT, SMT and the Z3 engines **33**
9.1 From SAT to SMT . . . 33
9.2 Z3 basics . . . 33
9.3 Proving by refutation . . . 34
9.4 The obstacle: transcendental functions . . . 34
9.5 Engine 1 (default): pointwise reasoning over CROWN's linear bounds . . . 34
9.6 Engine 2: full-graph relaxation (`z3_check_box`) . . . 35
9.6.1 Why engine 2 is not the default . . . 35
9.7 The spec-level helper . . . 36

10 Branch and bound **37**
10.1 The algorithm . . . 37
10.2 The per-box tests . . . 37
10.3 Correctness . . . 38
10.4 Complexity: the curse of dimensionality . . . 38
10.5 Implementation details (`BranchAndBoundVerifier.run`) . . . 38

11 The certificate and the proof checker **39**
11.1 What is stored . . . 39
11.2 The trusted computing base . . . 39
11.3 What the checker does (`ProofChecker.check`) . . . 39

Part IV — Synthesis **41**

12 Learning the controller and the barrier **42**
12.1 The learning problem . . . 42
12.2 Hinge losses with margins . . . 42
12.3 The loss terms (`CGISTrainer.losses`) . . . 42
12.4 The smooth band: why the Lie term uses a sigmoid . . . 43
12.5 Sampling strategy . . . 43
12.6 The controller smoothness penalty . . . 43
12.7 The nominal controller (car) . . . 43
12.8 Optimisation . . . 44

13 Counterexample-guided inductive synthesis (CGIS) **45**
13.1 Origin of the idea . . . 45
13.2 The loop (`CGISTrainer.run`) . . . 45
13.3 Design decisions . . . 45
13.4 Convergence . . . 46

Part V — Implementation **47**

14 Architecture and data flow **48**
14.1 Component diagram . . . 48
14.2 The “one function, many backends” principle . . . 48

15 Source code reference **49**
15.1 `src/ops.py`: backend dispatch . . . 49
15.2 `src/dynamics/continuous_system.py`: systems . . . 49
15.3 `src/models/`: networks . . . 50
15.4 `src/verification/interval.py` . . . 50
15.5 `src/verification/lirpa_verifier.py` . . . 50
15.6 `src/verification/smt_solver.py` . . . 51
15.7 `src/verification/cgis_loop.py` . . . 51
15.8 `src/verification/proof_checker.py` . . . 51
15.9 `src/visualization/phase_portrait.py` . . . 51
15.10 `scripts/` . . . 51

16 Configuration reference **53**
16.1 Top level . . . 53
16.2 `train.*` . . . 53
16.3 `verify.*` . . . 54
16.4 System parameters . . . 54

17 Running the project **55**
17.1 Installation . . . 55
17.2 Step-by-step tutorial . . . 55
17.3 Reading a round summary . . . 55

17.4 Troubleshooting . . . 56

18 Tests and the dashboard **57**
18.1 The test suite (`pytest`) . . . 57
18.2 The dashboard (`dashboard/app.py`) . . . 57

Part VI — Experiments **59**

19 Experimental setup **60**

20 Results **61**
20.1 Inverted pendulum: certified . . . 61
20.2 Dubins car: falsified (a partial result) . . . 62
20.3 Why the car fails: the zero-speed obstruction . . . 64

21 Development log: problems encountered and how they were solved **65**

Part VII — Perspective **67**

22 Related work, limitations and future work **68**
22.1 Related approaches . . . 68
22.2 Limitations . . . 68
22.3 Future work . . . 69

Appendix A — Notation **70**

Appendix B — Glossary **71**

Appendix C — Exercises with solutions **72**

Appendix D — Frequently asked questions **73**

Appendix E — References **74**

About this report

This report explains **everything** in the CertiControl-NN project, assuming nothing beyond first-year calculus, basic linear algebra and Python. It is written so that a reader who has never heard of barrier functions, bound propagation or SMT solvers can finish it able to:

- explain *why* the project exists and what exactly it proves;
- derive every equation used, including the proofs;
- read and modify every source file;
- run, reproduce and extend the experiments;
- judge the results critically, including their limitations.

How the report is organised.

Part	Chapters	Content
I. Background	1–4	motivation; mathematical prerequisites; dynamical systems; neural networks and differentiation
II. Safety theory	5	invariance, barrier certificates, the four properties, the main theorem with its full proof
III. Verification	6–11	soundness; interval arithmetic; linear relaxations and CROWN; SAT/SMT and Z3; branch and bound; proof checking
IV. Synthesis	12–13	training losses, and the counterexample-guided loop
V. Implementation	14–18	every file and function; configuration; running; tests; dashboard
VI. Experiments	19–21	setup, results, the full Dubins story, development lessons
VII. Perspective	22	related work, limitations, future work
Appendices	A–E	notation, glossary, exercises with solutions, FAQ, references

Each technical idea is presented in the same order: **intuition → theory (with proofs) → worked numerical example → code → design rationale** ("why this, and not something else").

Notation used throughout. Vectors are lower case (x), matrices upper case (W). x_i is the i-th component. $\leq$ between vectors is componentwise. $[\underline{a}, \overline{a}]$ denotes an interval; an underline means a lower bound and an overline an upper bound. $\odot$ is elementwise multiplication. $|W|$ is the elementwise absolute value. Code identifiers are in `monospace`. Appendix A has a full notation table.

Part I — Background

Chapter 1

Motivation: from testing to proof

1.1 The deployment problem

Neural networks trained by reinforcement learning or imitation learning are increasingly used as **controllers**: functions that map sensor readings to motor commands in cars, drones, robot arms and medical devices. Before deployment, such controllers are usually validated by **testing**: run many simulations and check that nothing bad happens.

Testing answers the question *"did anything bad happen on the cases I tried?"*. Safety certification needs the answer to *"can anything bad ever happen?"*. These questions differ fundamentally because:

1. **The state space is continuous.** A car's state (position, heading, speed) is a vector of real numbers. There are uncountably many states. Any finite test set covers a set of volume zero.
2. **Neural networks are not smooth in the engineering sense.** They can change behaviour sharply in small regions, especially regions that training rarely visited.
3. **Failures compound over time.** A small error at one state moves the system to a different state, where the controller may behave worse. Rare bad regions are reached by trajectories, not by independent random draws.

1.2 A concrete demonstration from this project

The 4-dimensional Dubins car experiment (Chapter 20) produced a controller/barrier pair with these properties:

evidence	verdict it suggests
2,000,000 uniformly random states: 23 violate the safety condition ($\approx 10^{-5}$)	"essentially safe"
100 simulated drives from safe states for 10 s: none leave the safe region	"safe"
the formal verifier: 524 validated violating states; certification refused	**not safe**
50 simulations started *at the verifier's counterexamples*: 41 leave the safe region	the verifier is right

This single table is the motivation for the whole project. The verifier found failures that testing effectively never finds, and it can prove their absence when they really are absent, as for the pendulum.

1.3 What the project builds

CertiControl-NN is an end-to-end system that:

1. **defines** a physical system with its safety requirements (Chapter 3);
2. **trains** two neural networks jointly: a controller π_θ and a barrier function B_ϕ (Chapter 12);
3. **verifies**, with mathematical rigour, that B_ϕ certifies safety of the closed loop *for all states* in a domain (Chapters 6–10);
4. if verification fails, **extracts concrete counterexamples**, adds them to the training data and repeats (Counterexample-Guided Inductive Synthesis, Chapter 13);
5. **stores** the proof and re-checks it with an independent checker (Chapter 11);
6. **visualises** everything in a Streamlit dashboard (Chapter 18).

It runs on a laptop CPU with open-source tools only: PyTorch, auto_LiRPA, Z3, SciPy, Hydra, Streamlit and Matplotlib.

1.4 Summary of results

system	state dim.	outcome
torque-limited inverted pendulum	2	**Certified** in 3 synthesis rounds; proof of 140 boxes, checked independently
Dubins car with obstacle and walls	4	**Not certified**: 99.4% of the domain volume proven, 524 genuine counterexamples found after 40 rounds

The second outcome is reported honestly as a partial result. Chapter 20 explains exactly why the failures occur.

Chapter 2

Mathematical prerequisites

This chapter collects every piece of mathematics used later. Skim it if it is familiar.

2.1 Vectors, norms and boxes

- A vector $x \in \mathbb{R}^n$ has components $x_1, \dots, x_n$.
- **Norms** measure size: $\|x\|_2 = \sqrt{\sum_i x_i^2}$ (Euclidean) and $\|x\|_\infty = \max_i |x_i|$ (maximum norm).
- A **box** (hyper-rectangle) is $R = [l, u] = \{x : l_i \le x_i \le u_i \ \forall i\}$. In centre–radius form, $c = \frac{l+u}{2}$ and $r = \frac{u-l}{2}$, so $R = \{x : \|x - c\|_\infty \le r\}$ when all r_i are equal. This is why auto_LiRPA calls a box an "L_∞ perturbation".
- **Volume** of a box: $\prod_i (u_i - l_i)$. The proof checker uses volumes to confirm that the proof boxes cover the domain.

2.2 Minimising a linear function over a box

This small fact is used everywhere in Chapters 7–10.

Lemma 2.1. For $a \in \mathbb{R}^n$ and $R = [l, u]$:

$$\min_{x \in R} a^\top x = \sum_i \min(a_i l_i,\, a_i u_i) = a^\top c - |a|^\top r.$$

Proof. The objective is a sum of independent one-dimensional terms. Each term $a_i x_i$ is minimised at $x_i = l_i$ if $a_i \ge 0$ and at $x_i = u_i$ otherwise. Writing $x_i = c_i + t_i r_i$ with $t_i \in [-1, 1]$ gives $a_i c_i + a_i r_i t_i$, whose minimum is $a_i c_i - |a_i| r_i$. $\square$

Similarly, $\max_{x \in R} a^\top x = a^\top c + |a|^\top r$. This operation is called **concretization**.

2.3 Derivatives, gradients and Jacobians

- For $h : \mathbb{R}^n \to \mathbb{R}$, the **gradient** $\nabla h(x)$ is the vector of partial derivatives $\partial h / \partial x_i$.
- For $F : \mathbb{R}^n \to \mathbb{R}^m$, the **Jacobian** $J_F(x)$ is the $m \times n$ matrix $[\partial F_i / \partial x_j]$.
- **Chain rule.** $J_{G \circ F}(x) = J_G(F(x))\, J_F(x)$. Along a curve $x(t)$: $\frac{d}{dt} h(x(t)) = \nabla h(x(t)) \cdot \dot{x}(t)$.
- **Directional derivative.** The derivative of h at x in direction v is $\nabla h(x) \cdot v = \lim_{\epsilon \to 0} \frac{h(x + \epsilon v) - h(x)}{\epsilon}$. For vector-valued maps it is the **Jacobian-vector product (JVP)** $J_F(x)\, v$.

2.4 The mean value theorem and Lipschitz continuity

Mean value theorem (MVT). If $f : [l, u] \to \mathbb{R}$ is differentiable, then for $x, y \in [l, u]$ there is a ξ between them with $f(y) - f(x) = f'(\xi)(y - x)$.

Consequence used by the Z3 encoding (Chapter 9). Suppose $d_{\min} \le f' \le d_{\max}$ on $[l, u]$. Then for $x \in [l, u]$:

$$f(l) + d_{\min}(x - l) \le f(x) \le f(l) + d_{\max}(x - l), \qquad f(u) - d_{\max}(u - x) \le f(x) \le f(u) - d_{\min}(u - x).$$

Proof. Apply the MVT to (l, x): $f(x) - f(l) = f'(\xi)(x - l)$ with $x - l \ge 0$, so the difference lies between $d_{\min}(x - l)$ and $d_{\max}(x - l)$. Apply it to (x, u) in the same way. □

A function is **L-Lipschitz** if $|h(x) - h(y)| \le L\|x - y\|$. Lipschitz constants bound how much a function can change within a box. That is why smooth controllers (small L) are easier to verify (Chapter 12).

2.5 Convexity and concavity

f is **convex** on an interval if chords lie above the graph, and **concave** if chords lie below. For a twice differentiable f, convexity means $f'' \ge 0$. tanh is convex for $z \le 0$ and concave for $z \ge 0$, because $\tanh''(z) = -2\tanh(z)(1 - \tanh^2 z)$. This decides which lines give valid linear bounds (Chapter 8).

2.6 Sets: interior, boundary, compactness

- The **boundary** ∂S is the set of points arbitrarily close to both S and its complement. The **interior** is $S \setminus \partial S$ (for closed S).
- A set in $\mathbb{R}^n$ is **compact** if it is closed and bounded. Continuous functions attain their minimum and maximum on compact sets, and solutions of ODEs that stay in a compact set exist for all time.
- The **super-level set** of B is $\{x : B(x) \ge 0\}$. It is closed if B is continuous.

2.7 Linear programming (LP) in one paragraph

An LP minimises a linear function subject to linear inequalities. LPs are solvable in polynomial time. A set of linear inequalities is a **polyhedron**. Chapter 9 uses the fact that deciding whether a polyhedron is empty (*feasibility*) is exactly what an SMT solver for linear real arithmetic does.

2.8 Floating-point arithmetic

Computers represent reals as IEEE-754 doubles (float64): 53 significant bits. Every operation rounds to the nearest representable number, with relative error at most the **unit roundoff** $u = 2^{-53} \approx 1.1 \times 10^{-16}$. The code uses the more conservative 2^{-52}. For a dot product of n terms, the computed result $\hat{s}$ satisfies the classic bound (Higham, *Accuracy and Stability of Numerical Algorithms*):

$$|\hat{s} - s| \le \gamma_n \sum_i |a_i b_i|, \qquad \gamma_n = \frac{nu}{1 - nu} \approx nu.$$

A sound verifier must not let such errors turn “almost true” into “proven”. Chapter 7 explains how the interval code accounts for them.

Chapter 3

Dynamical systems and control

3.1 States and ordinary differential equations

The **state** $x(t) \in \mathbb{R}^n$ is the information needed to predict the future. A continuous-time system evolves by an **ODE** $\dot{x} = F(x)$.

Existence and uniqueness (Picard–Lindelöf). If F is locally Lipschitz, then for each x_0 there is a unique solution on some time interval $[0, T)$. If the solution stays in a compact set, it exists for all $t \geq 0$. All functions in this project (polynomials, sin, cos, tanh networks) are smooth, hence locally Lipschitz.

3.2 Controlled systems and the control-affine form

With an input u the system becomes $\dot{x} = F(x, u)$. The project uses the **control-affine** form

$$\dot{x} = f(x) + g(x)\, u, \qquad u \in \mathcal{U} = [u_{\min}, u_{\max}] \subset \mathbb{R}^m.$$

Why control-affine?

1. It covers most mechanical systems: forces and torques enter linearly through Newton's law.
2. Barrier-function theory is simplest in this setting: $\dot{B}$ is affine in u.
3. The closed-loop computation is a small graph that bounds tightly.

Code: one definition, two derived objects. Each system implements only `xdot(x, u)`. The drift and input matrix are derived (`ControlAffineSystem.f, .g`):

$$f(x) = \texttt{xdot}(x, 0), \qquad g(x)_{:,j} = \texttt{xdot}(x, e_j) - f(x).$$

This is exact because `xdot` is affine in u. It guarantees that f, g and `xdot` can never disagree.

3.3 System 1: the torque-limited inverted pendulum

3.3.1 Derivation from Newton's law

A point mass m on a massless rod of length l rotates about a pivot. Let θ be the angle from upright. Gravity produces torque $mgl \sin\theta$, pushing away from upright. Joint friction produces torque $-b\dot{\theta}$, and the motor produces torque u. The moment of inertia is $I = ml^2$. Newton's law for rotation, $I\ddot{\theta} = \sum \text{torques}$, gives

$$ml^2\ddot{\theta} = mgl\sin\theta - b\dot{\theta} + u \quad \Longrightarrow \quad \ddot{\theta} = \frac{g}{l}\sin\theta - \frac{b}{ml^2}\dot{\theta} + \frac{1}{ml^2}u.$$

With state $x = (\theta, \omega)$, $\omega = \dot{\theta}$:

$$f(x) = \begin{pmatrix} \omega \\ \frac{g}{l}\sin\theta - \frac{b}{ml^2}\omega \end{pmatrix}, \qquad g(x) = \begin{pmatrix} 0 \\ \frac{1}{ml^2} \end{pmatrix}.$$

3.3.2 Parameters and sets

symbol	value	meaning
m, l	1, 1	mass, length
b	0.1	friction
g	9.81	gravity
$u_{\max}$	6	torque limit, $\lvert u\rvert \le 6$
D	$[-1,1]\times[-4,4]$	verification domain
X_{unsafe}	$\lvert\theta\rvert \ge 0.8$	too far from upright
X_0	$[-0.25,0.25]\times[-0.4,0.4]$	initial states

Why the problem is non-trivial. To *hold* the pendulum at angle θ takes torque $mgl\sin\theta = 9.81\sin\theta$. With $|u| \le 6$, holding is impossible beyond $\sin\theta = 6/9.81$, that is $\theta \approx 0.66$ rad, which is less than the unsafe threshold 0.8. Moving states are harder still: a pendulum at $\theta = 0.5$ swinging outward fast cannot be stopped in time. The largest region from which safety can be maintained, the **viability kernel**, is therefore a tilted, lens-shaped region that nobody wrote down in advance. The training must discover it.

3.3.3 Code

```
class InvertedPendulum(ControlAffineSystem):
    def xdot(self, x, u):
        th, om = x[:, 0:1], x[:, 1:2]           # columns kept 2-D: shape (N,1)
        ml2 = self.m * self.l ** 2
        dom = (self.grav / self.l) * ops.sin(th) + (-self.b / ml2) * om + (1.0 / ml2) * u[:, 0:1]
        return ops.cat([om, dom])               # shape (N,2)

    def features(self, x):                      # psi(x) = (theta, omega/4)
        return ops.cat([x[:, 0:1], (1.0 / self.omega_scale) * x[:, 1:2]])

    def features_jvp(self, x, xdot):            # d/dt psi(x) = J_psi(x) xdot
        return ops.cat([xdot[:, 0:1], (1.0 / self.omega_scale) * xdot[:, 1:2]])

    def unsafe_pieces(self, x):                 # unsafe = {0.8-theta<=0} U {theta+0.8<=0}
        th = x[:, 0:1]
        return [(-1.0 * th) + self.theta_unsafe, th + self.theta_unsafe]
```

Design notes.

- Slicing with `0:1` instead of `0` keeps shapes `(N,1)`, so the same code works for tensors, intervals and Z3 vectors.
- The unsafe set is written as a **union of sub-level sets** $\bigcup_i \{h_i(x) \le 0\}$. This format is simple to test with intervals (can $h_i \le 0$ happen in this box?) and to encode exactly in Z3 (a disjunction).
- The feature $\omega/4$ rescales $\omega \in [-4,4]$ to $[-1,1]$, the same range as θ. Networks train better when inputs have similar scales.

3.4 System 2: the Dubins car with acceleration

3.4.1 Kinematics

A car moves in the plane with position (p_x, p_y), heading θ and forward speed v (negative v means reversing). Moving at speed v in direction θ gives $\dot{p}_x = v\cos\theta$ and $\dot{p}_y = v\sin\theta$. The inputs are the turn rate

$u_1 = \dot{\theta}$ and the acceleration $u_2 = \dot{v}$:

$$f(x) = \begin{pmatrix} v\cos\theta \\ v\sin\theta \\ 0 \\ 0 \end{pmatrix}, \qquad g(x) = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

item	value
domain D	$p_x, p_y \in [-2, 2]$ (walls), $\theta \in [-\pi, \pi]$ (periodic), $v \in [-1, 1]$
inputs	$\lvert u_1\rvert \le 2$ rad/s, $\lvert u_2\rvert \le 2$ m/s²
unsafe set	obstacle disc $p_x^2 + p_y^2 \le 0.6^2$
initial set X_0	$\lvert p_x\rvert, \lvert p_y\rvert \le 1.4$, $\lvert v\rvert \le 0.25$, any heading, at least 0.35 from the obstacle surface
nominal task	drive towards goal $(1.4, 1.4)$ at speed 0.5 (a soft preference only)

Walls and speed limits are not in the unsafe set. Instead, they are faces of D that the safe set must never touch (property P3, Chapter 5). The safe set must therefore also keep the car away from walls and prevent it from exceeding $\lvert v\rvert = 1$.

3.4.2 The periodic heading

$\theta = \pi$ and $\theta = -\pi$ describe the same physical state. If the networks saw θ directly, they could assign different values to the two, and the domain would have artificial faces at $\theta = \pm\pi$. The **feature map** therefore uses $(\cos\theta, \sin\theta)$:

$$\psi(x) = (p_x/2,\ p_y/2,\ \cos\theta,\ \sin\theta,\ v),$$

and θ is declared periodic (`periodic=[2]`) so that no face is created for it. The time derivative of the features, needed for $\dot{B}$, is derived with the chain rule:

$$\frac{d}{dt}\psi = (\dot{p}_x/2,\ \dot{p}_y/2,\ -\sin\theta\,\dot{\theta},\ \cos\theta\,\dot{\theta},\ \dot{v}).$$

```
def features_jvp(self, x, xdot):
    s = 1.0 / self.arena
    th, dth = x[:, 2:3], xdot[:, 2:3]
    return ops.cat([s * xdot[:, 0:1], s * xdot[:, 1:2],
                    (-1.0 * ops.sin(th)) * dth, ops.cos(th) * dth, xdot[:, 3:4]])
```

3.4.3 Sets written as polynomials

```
def unsafe_pieces(self, x):     # (px-cx)^2 + (py-cy)^2 - r^2 <= 0
    dx = x[:, 0:1] + (-self.c[0]); dy = x[:, 1:2] + (-self.c[1])
    return [_square(dx) + _square(dy) + (-self.r ** 2)]

def init_constraints(self, x):  # X0 also requires distance >= r + 0.35
    ...
    return [_square(dx) + _square(dy) + (-(self.r + self.init_clearance) ** 2)]
```

Z3 can decide polynomial constraints exactly, so the obstacle is encoded **without approximation** (Chapter 9).

3.5 Feedback, closed loop and simulation

A controller $u = \pi(x)$ produces the **closed loop** $\dot{x} = F_{cl}(x) = f(x) + g(x)\pi(x)$.

Simulation (`src/visualization/phase_portrait.py::simulate`) integrates the closed loop with SciPy's `solve_ivp` using **RK45** (Dormand–Prince). This method estimates its own error by comparing a 4th- and a 5th-order Runge–Kutta step, and adapts the step size so the error stays below the tolerances (`rtol=1e-7`, `atol=1e-9`). Compare the simplest method, explicit Euler $x_{k+1} = x_k + hF(x_k)$, whose error is $O(h)$. RK45 is far more accurate at the same cost. Simulation in this project is for **illustration and cross-checking only**; it never contributes to a proof.

Chapter 4

Neural networks and differentiation

4.1 The multilayer perceptron

An MLP with hidden widths $n_1, \dots, n_{L-1}$ computes

$$h_0 = s, \qquad z_k = W_k h_{k-1} + b_k, \quad h_k = \tanh(z_k)\ (k < L), \qquad y = z_L = W_L h_{L-1} + b_L.$$

Parameter counts (hidden widths [32, 32]):

network	input dim	parameters
pendulum controller	2	$(2 \cdot 32 + 32) + (32 \cdot 32 + 32) + (32 \cdot 1 + 1) = 1185$
pendulum barrier	2	1185
Dubins controller	5	$(5 \cdot 32 + 32) + (32 \cdot 32 + 32) + (32 \cdot 2 + 2) = 1314$
Dubins barrier	5	1281

These are deliberately **small**. Verification cost grows quickly with network size, and small networks already have enough capacity for these problems.

`src/models/neural_barrier.py::build_tanh_mlp` builds the layers as a `ModuleList` of `nn.Linear`.

4.2 Why tanh?

$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ has these properties:

1. **Smoothness.** $\tanh'(z) = 1 - \tanh^2 z$ and $\tanh''(z) = -2 \tanh z\,(1 - \tanh^2 z)$. The barrier condition uses ∇B, which must exist everywhere.
2. **Monotone and bounded**, with range $(-1, 1)$. Interval bounds are trivial: $\tanh[\underline{z}, \overline{z}] = [\tanh \underline{z}, \tanh \overline{z}]$.
3. $\tanh'$ **is bounded, positive and unimodal**, with maximum 1 at $z = 0$. This gives cheap, tight bounds on derivatives.

ReLU comparison. $\mathrm{ReLU}(z) = \max(0, z)$ is standard in vision and simple to verify for plain outputs. But B would be piecewise linear and its gradient would jump. $\dot{B}$ would then be undefined on the kinks, and invariance arguments would need nonsmooth analysis (Clarke generalized gradients). Smooth activations avoid all of this.

4.3 The controller: saturation by construction

$$\pi(x) = u_{\text{mid}} + u_{\text{half}} \odot \tanh(\mathrm{MLP}_\theta(\psi(x))).$$

Since tanh $\in (-1, 1)$, every output component lies in $(u_{\min}, u_{\max})$ **for every weight setting and every input**. Actuator limits therefore never need to be verified, and the verifier's interval bounds automatically respect them. `test_controller_respects_actuator_limits` multiplies all weights by 50 and confirms the limits still hold.

4.4 Automatic differentiation

Automatic differentiation (AD) computes exact derivatives of programs by applying the chain rule to every elementary operation.

4.4.1 Reverse mode

Reverse mode (backpropagation) runs the program forward, then propagates **adjoints** $\bar{v} = \partial y/\partial v$ backward. One backward pass gives the full gradient of a scalar output, at a cost of a small constant times the forward pass. PyTorch's `loss.backward()` uses it for training.

4.4.2 Forward mode and dual numbers

Forward mode propagates **tangents** $\dot{v} = \frac{\partial v}{\partial x} \cdot d$ alongside the values, for a chosen direction d. Algebraically this is arithmetic on **dual numbers** $a + b\varepsilon$ with $\varepsilon^2 = 0$:

$$f(a + b\varepsilon) = f(a) + f'(a)b\,\varepsilon.$$

One forward pass gives one directional derivative (JVP).

4.4.3 Why forward mode fits this project exactly

The barrier condition needs

$$\dot{B}(x) = \nabla B(x) \cdot F_{cl}(x),$$

which is precisely the directional derivative of B in direction $F_{cl}(x)$. A single forward-mode pass with seed direction $F_{cl}(x)$ computes it. The result is an ordinary feed-forward graph that bound propagation tools can analyse. Reverse mode would give the same number, but through a backward graph. Tools like auto_LiRPA are designed around forward computation graphs, and bounding a gradient computed by backpropagation would be far more involved.

4.4.4 Derivation of the tangent recursion

Let $d\psi = J_\psi(x)F_{cl}(x)$ be the feature-space direction. For the affine layer $z = Wh + b$, the chain rule gives $dz = W\,dh$ (the constant b contributes nothing). For $h' = \tanh(z)$, applied elementwise, it gives $dh' = \tanh'(z) \odot dz$. So:

$$\begin{aligned}
(h_0,\, dh_0) &= (\psi(x),\, J_\psi F_{cl}) \\
(z_k,\, dz_k) &= (W_k h_{k-1} + b_k,\, W_k\, dh_{k-1}) \\
(h_k,\, dh_k) &= (\tanh z_k,\, (1 - h_k^2) \odot dz_k) \\
(B,\, \dot{B}) &= (z_L,\, dz_L)
\end{aligned}$$

4.4.5 Worked example

Take a one-neuron barrier $B(x) = 2\tanh(x_1 - x_2)$ and closed-loop field $F_{cl}(x) = (x_2, -x_1)$ at $x = (1, 0.5)$.

- Value: $z = 0.5$, $h = \tanh 0.5 = 0.4621$, $B = 0.9242$.
- Tangent: the seed is $F_{cl} = (0.5, -1)$. Then $dz = 1 \cdot 0.5 + (-1)(-1) = 1.5$, $dh = (1 - 0.4621^2) \cdot 1.5 = 1.1797$, and $\dot{B} = 2 \cdot 1.1797 = 2.3594$.
- Check by the gradient: $\nabla B = 2(1 - h^2)(1, -1) = (1.5729, -1.5729)$, and $\nabla B \cdot F_{cl} = 0.7864 + 1.5729 = 2.3594$. □

4.4.6 Code

```
def mlp_forward(layers, h, dh=None):
    for i, layer in enumerate(layers):
        z = ops.linear(h, layer.weight, layer.bias)              # z = W h + b
        dz = ops.linear(dh, layer.weight, None) if dh is not None else None   # dz = W dh
        if i == len(layers) - 1:
            return z, dz                                         # output layer: no tanh
        h = ops.tanh(z)
        if dh is not None:
            dh = ops.dtanh(z, h) * dz                            # (1-h^2) * dz
```

`ops.dtanh(z, h)` returns `1 - h*h` for tensors. For intervals and Z3, it uses z directly to obtain tighter bounds (Chapter 7).

The certificate module (`src/models/certificate.py`) chains everything:

```
def terms(self, x):
    s = self.system.features(x)                 # psi(x)
    u = self.controller(s)                      # pi(x)
    xdot = self.system.xdot(x, u)               # F_cl(x) = f + g pi
    ds = self.system.features_jvp(x, xdot)      # J_psi(x) F_cl(x)
    B, dB = self.barrier.forward_with_tangent(s, ds)
    return B, dB, u

def forward(self, x):
    B, dB, _ = self.terms(x)
    return ops.cat([B, dB + self.gamma * B])    # the two verified outputs
```

Validation. `test_certificate_lie_derivative_matches_autograd` computes ∇B with reverse-mode autograd, forms $\nabla B \cdot (f+g\pi)+\gamma B$ independently, and checks agreement to 10^{-10} for both systems. The spec's reference method `NeuralControlBarrier.compute_lie_derivative` (reverse mode) is also tested against the forward-mode tangent.

4.5 One computation, several meanings: `src/ops.py`

Every verified computation is written in terms of seven primitives plus `+ - *` and slicing:

primitive	meaning
`sin, cos, tanh`	elementwise functions
`dtanh(z, h)`	$1 - \tanh^2(z)$
`linear(x, W, b)`	$xW^\top + b$
`scale_shift(x, a, c)`	$x \odot a + c$ with constant vectors
`cat(xs)`	concatenate features

Each primitive checks the type of its argument:

```
def tanh(x):
    return torch.tanh(x) if _is_tensor(x) else x.tanh()
```

When `x` is an `Interval`, `x.tanh()` returns a sound interval. When `x` is a `Z3RelaxedVec`, it returns fresh Z3 variables with enclosing constraints. Python's operator overloading (`__add__`, `__mul__`, `__getitem__`) handles the rest.

Why this design matters. It guarantees *the verified function is the trained function*. There is only one implementation of the dynamics, features, networks and Lie derivative. A typical bug in verification projects, a formula in the verifier that differs slightly from the model in the trainer, becomes impossible by construction.

Part II — Safety theory

Chapter 5

Invariance, barrier certificates and the main theorem

5.1 Safety as "never enter the unsafe set"

Given a closed-loop system $\dot{x} = F_{cl}(x)$, an **unsafe set** X_{unsafe} and a set of **initial states** X_0, the system is *safe* if every trajectory with $x(0) \in X_0$ satisfies $x(t) \notin X_{\text{unsafe}}$ for all $t \geq 0$.

Checking this directly means reasoning about infinitely many infinitely long trajectories. The standard trick is to reduce it to a **local** condition on a set.

5.2 Forward invariance

A set $\mathcal{C}$ is **forward invariant** if $x(0) \in \mathcal{C}$ implies $x(t) \in \mathcal{C}$ for all $t \geq 0$.

Safety via invariance. If some $\mathcal{C}$ satisfies

(a) $X_0 \subseteq \mathcal{C}$,

(b) $\mathcal{C} \cap X_{\text{unsafe}} = \emptyset$, and

(c) $\mathcal{C}$ is forward invariant,

then the system is safe. *Proof:* a trajectory starting in X_0 starts in $\mathcal{C}$ by (a), stays in $\mathcal{C}$ by (c), and $\mathcal{C}$ contains no unsafe state by (b). □

The remaining question is how to establish (c) without simulating.

5.3 Describing sets by functions

Let $\mathcal{C} = \{x : B(x) \geq 0\}$ for a continuously differentiable $B : \mathbb{R}^n \to \mathbb{R}$. Along a trajectory,

$$\dot{B}(x) = \frac{d}{dt} B(x(t)) = \nabla B(x) \cdot F_{cl}(x) =: L_{F_{cl}} B(x),$$

the **Lie derivative** of B along F_{cl}. It is a function of the state only, so it can be checked pointwise, and hence over boxes, without simulating.

5.4 Nagumo's theorem

Theorem (Nagumo, 1942; informal). Let $\mathcal{C}$ be closed and assume solutions are unique. Then $\mathcal{C}$ is forward invariant if and only if, at every boundary point, $F_{cl}(x)$ points into $\mathcal{C}$ or is tangent to it (it lies in the *tangent cone*).

For $\mathcal{C} = \{B \geq 0\}$ with $\nabla B \neq 0$ on the boundary, the condition reads $\dot{B}(x) \geq 0$ wherever $B(x) = 0$. This is intuitive: on the boundary, B must not start to decrease. The figure below illustrates this.

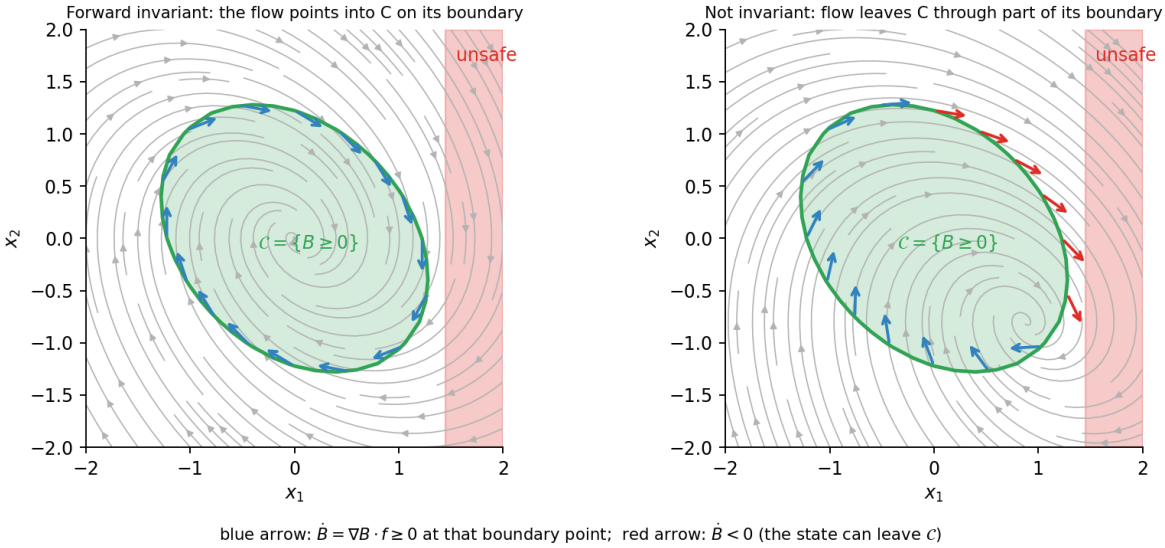


Figure 5.1: Left: the flow points into $\mathcal{C}$ everywhere on its boundary, so $\mathcal{C}$ is forward invariant. Right: on part of the boundary (red arrows) $\dot{B} < 0$, and trajectories escape towards the unsafe region.

Why not check just the boundary? Two reasons. First, the boundary is a thin, lower-dimensional set: a box verifier cannot isolate it, and any box touching the boundary also contains points just inside and just outside. Second, the condition "≥ 0 exactly at $B = 0$" has no robustness margin: a tiny numerical error flips it. Both problems are fixed by strengthening the condition to all of $\mathcal{C}$.

5.5 Barrier certificates and control barrier functions

Barrier certificates (Prajna & Jadbabaie, 2004) are functions separating initial and unsafe sets with a sign condition on $\dot{B}$. **Control barrier functions** (Ames et al., 2017/2019) generalise this to systems with inputs. The standard *zeroing CBF* condition is

$$\dot{B}(x) \;\geq\; -\alpha(B(x)) \qquad \text{for all } x \in \mathcal{C},$$

where α is an (extended) **class-$\mathcal{K}$ function**: continuous, strictly increasing, with $\alpha(0) = 0$.

Interpretation. On the boundary ($B = 0$) the condition becomes $\dot{B} \geq 0$, which is Nagumo's condition. Inside, B may decrease, but no faster than $\alpha(B)$. As the state approaches the boundary, the permitted rate of decrease goes to zero, like a car braking harder the closer it gets to a wall.

This project uses $\alpha(y) = \gamma y$ with $\gamma = 1$. The quantity checked everywhere is

$$\boxed{\text{cond}(x) = \nabla B(x) \cdot \big(f(x) + g(x)\pi(x)\big) + \gamma B(x) \;\geq 0}$$

Neural CBF. Here $B = B_\phi$ is a neural network and the input is fixed to the neural controller π_θ. Both are learned. Because the controller is fixed inside the proof, the certificate says: *this specific closed loop is safe*. There is no online optimisation, unlike a CBF-QP safety filter, which solves a quadratic program at every control step.

5.6 The comparison lemma, with proof

Lemma 5.1. Let $b : [0, T) \to \mathbb{R}$ be continuously differentiable with $b(0) \geq 0$, and suppose $\dot{b}(t) + \gamma b(t) \geq \tau$ whenever $b(t) \geq 0$, for some $\tau > 0$ (the verified margin). Then $b(t) \geq e^{-\gamma t} b(0) \geq 0$ for all $t \in [0, T)$.

Proof.

Step 1: b never becomes negative. Suppose it does, and let $t_0 = \inf\{t : b(t) < 0\}$. By continuity $b(t_0) = 0$, and b takes negative values at times arbitrarily close after t_0. But at t_0 the hypothesis applies: $\dot{b}(t_0) \geq \tau - \gamma \cdot 0 = \tau > 0$. So b is strictly increasing at t_0, and $b > 0$ on some interval $(t_0, t_0 + \delta)$. This contradicts the choice of t_0. Hence $b(t) \geq 0$ for all t.

Step 2: the exponential bound. Since $b \geq 0$ everywhere, the hypothesis holds at all times. Let $\phi(t) = e^{\gamma t} b(t)$. Then $\dot{\phi} = e^{\gamma t}(\dot{b} + \gamma b) \geq e^{\gamma t} \tau \geq 0$, so ϕ is non-decreasing: $e^{\gamma t} b(t) \geq b(0)$, that is, $b(t) \geq e^{-\gamma t} b(0)$. □

Remark. The margin $\tau = 10^{-8}$ enforced by the verifier (Chapter 7) is not only a guard against rounding. It also makes Step 1 a one-line argument. Without a margin ($\tau = 0$), the same conclusion needs a more careful argument (for example, Nagumo's theorem with uniqueness of solutions).

5.7 The four certified properties

The verifier works on a bounded box D (the domain). Define $\mathcal{C} = \{x \in D : B(x) \geq 0\}$. The certified properties are:

	statement	why it is needed
P1 invariance	$\forall x \in D :\ B(x) \geq 0 \Rightarrow \text{cond}(x) \geq \tau$	the CBF condition on the whole safe set
P2 safety	$\forall x \in D \cap X_{\text{unsafe}} :\ B(x) < 0$	ensures $\mathcal{C} \cap X_{\text{unsafe}} = \emptyset$
P3 containment	$\forall x \in \partial D$ (non-periodic faces): $B(x) < 0$	ensures $\mathcal{C}$ does not reach the edge of the verified domain
P0 non-triviality	$\forall x \in X_0 :\ B(x) > 0$	ensures $X_0 \subseteq \mathcal{C}$, and rules out $\mathcal{C} = \emptyset$

What happens without each property.

- **Without P1:** nothing prevents leaving $\mathcal{C}$.
- **Without P2:** $\mathcal{C}$ might contain unsafe states.
- **Without P3:** the proof only covers states inside D. A trajectory could exit D through a face while B is still positive, enter the unverified region, and come back anywhere. Requiring $B < 0$ on the faces makes $\{B \geq 0\} \cap D$ a set that cannot touch ∂D. By continuity, any trajectory leaving D would first have to cross $B = 0$ inside D, which P1 forbids.
- **Without P0:** the constant function $B \equiv -1$ satisfies P1 (vacuously, since $B \geq 0$ never holds), P2 and P3. It "certifies" nothing. This really happened during development (Chapter 21): training converged to $B < 0$ everywhere until P0 was added to the verifier and weighted strongly in the loss.

5.8 Main theorem, with proof

Theorem 5.2. Let B and π be continuously differentiable, and suppose P0–P3 hold. Then every closed-loop trajectory with $x(0) \in X_0$ exists for all $t \geq 0$, stays in $\mathcal{C}$, and never enters X_{unsafe}.

Proof.

1. *$\mathcal{C}$ is compact and lies in the interior of D (in non-periodic coordinates).* $\mathcal{C}$ is closed (B is continuous) and bounded ($\subseteq D$). By P3, $B < 0$ on every non-periodic face of D, so $\mathcal{C}$ contains no point of those faces. For the periodic coordinate θ, the faces $\theta = \pm\pi$ are identified (the networks see $\cos\theta, \sin\theta$), so they are not a real boundary.
2. *Existence.* F_{cl} is locally Lipschitz, so a unique solution exists on some $[0, T)$. As long as it stays in the compact $\mathcal{C}$, it can be extended. So it suffices to show that it stays in $\mathcal{C}$.

3. *Invariance.* $x(0) \in X_0 \subseteq \mathcal{C}$ by P0. Let $b(t) = B(x(t))$. While $x(t) \in D$ and $b(t) \geq 0$, P1 gives $\dot{b} + \gamma b = \text{cond} \geq \tau > 0$. By Lemma 5.1, b never becomes negative, *provided* the trajectory stays in D. It cannot leave D while $b \geq 0$: to reach a face, $x(t)$ would have to arrive at a point where $B < 0$ (P3), so b would first have to cross zero inside D, which Lemma 5.1 excludes. Hence $x(t) \in \mathcal{C}$ for all t.
4. *Safety.* By P2, no point of $\mathcal{C}$ is unsafe. □

Corollary 5.3 (robustness in γ). If P1 holds for γ, it holds for every $\gamma' \geq \gamma$: on $\mathcal{C}$, $B \geq 0$, so $\gamma' B \geq \gamma B$. The dashboard exploits this. The pendulum proof was in fact re-verified successfully for $\gamma \in \{0.25, 0.5, 1, 2, 4\}$.

5.9 Connection to Lyapunov functions

A **Lyapunov function** V proves *stability* (convergence to an equilibrium) through $\dot{V} < 0$. A barrier function proves *invariance* (staying in a set) through $\dot{B} \geq -\alpha(B)$. Both reduce a statement about trajectories to a pointwise inequality on the state space, which is exactly what makes them verifiable by the same bound-propagation machinery. Neural Lyapunov control (Chang et al., 2019) and FOSSIL (Abate et al., 2021) apply the learn-then-verify idea to Lyapunov functions.

5.10 Why *learn* B and π?

Classical constructions (sum-of-squares programming, Hamilton–Jacobi reachability) scale poorly or require polynomial dynamics. Neural networks are flexible function approximators, and training can shape B to fit the true viability kernel. The price is that the result must be **verified**, because training only enforces the conditions at sampled points. The rest of the report is about paying that price rigorously.

Part III — Verification

Chapter 6

Verification fundamentals

6.1 The verification problem

After training, we must decide statements of the form

$$\forall x \in R : \; h(x) \geq 0,$$

where R is a box and h is built from the networks and the dynamics, for example $h = \text{cond}$. There are uncountably many x, so we cannot check them one by one. We need **symbolic** reasoning about whole sets.

6.2 Soundness and completeness

- **Sound**: if the verifier says "proven", the statement is true. *A verifier that is not sound is worse than none.*
- **Complete**: every true statement can be proven.
- **Decidability and cost.** Even for ReLU networks, exact verification is NP-complete (Katz et al., *Reluplex*, 2017). For networks with tanh and sin, exact reasoning is not decidable in general, because these are transcendental functions.

CertiControl-NN is **sound and incomplete**. It returns one of three verdicts:

verdict	meaning	strength
CERTIFIED	all properties proven on every box	a proof
FALSIFIED	at least one concrete state violates a property, checked on the real network	a disproof
UNKNOWN	neither, within the computing budget	no conclusion

6.3 Over-approximation: the central idea

An **enclosure** of h on R is a pair $\underline{h}, \overline{h}$ with $\underline{h} \leq h(x) \leq \overline{h}$ for all $x \in R$. Then:

- if $\underline{h} \geq 0$, then $h \geq 0$ on all of R: **proven**;
- if some concrete $x \in R$ has $h(x) < 0$: **counterexample**;
- otherwise the enclosure is too loose: **split** R and try again.

Enclosures get tighter as boxes shrink. Chapter 10 turns this into a complete algorithm, branch and bound. Chapters 7–9 are about computing enclosures: cheaply (intervals), tightly (linear relaxations), and pointwise (SMT).

6.4 Falsification is easy; proof is hard

Checking one point is trivial: evaluate the network. Every verifier in this project therefore **tries falsification first**, at the centre of each box and at one random point. Real violations are found cheaply, and the expensive bounding work is spent only on boxes that might be provable.

Chapter 7

Interval arithmetic and interval bound propagation (IBP)

7.1 Intuition

Replace each number by an interval guaranteed to contain it. Define interval versions of each operation that return intervals guaranteed to contain every possible result. Push the input box through the whole computation. The final interval is an enclosure.

7.2 Definitions and the fundamental theorem

An interval is $\mathbf{a} = [\underline{a}, \overline{a}]$. For an operation $\circ$, the interval extension must satisfy the **inclusion property**:

$$a \in \mathbf{a},\ b \in \mathbf{b} \ \Rightarrow\ a \circ b \in \mathbf{a} \circledcirc \mathbf{b}.$$

Fundamental theorem of interval arithmetic. If every elementary operation satisfies the inclusion property, then evaluating a composite expression in interval arithmetic produces an enclosure of the expression's range.

Proof sketch. By induction on the expression tree: each intermediate interval contains every value the corresponding sub-expression takes. □

7.3 The rules, with justification

operation	interval result	why it is correct
$a + b$	$[\underline{a} + \underline{b},\ \overline{a} + \overline{b}]$	addition is monotone in each argument
$a - b$	$[\underline{a} - \overline{b},\ \overline{a} - \underline{b}]$	$-b$ reverses the order
$c \cdot a$ (constant c)	$[c\underline{a}, c\overline{a}]$ if $c \geq 0$, else swapped	linear
$a \cdot b$	$[\min P, \max P]$, $P = \{\underline{a}\underline{b}, \underline{a}\overline{b}, \overline{a}\underline{b}, \overline{a}\overline{b}\}$	bilinear: extremes occur at corners
a^2	$[0, \max(\underline{a}^2, \overline{a}^2)]$ if $0 \in \mathbf{a}$, else $[\min, \max]$ of the endpoint squares	tighter than $a \cdot a$
$\tanh a$	$[\tanh \underline{a}, \tanh \overline{a}]$	monotone increasing
$Wh + b$	$Wc + b \pm \lvert W \rvert r$	Lemma 2.1, applied row by row

7.3.1 The affine layer in detail

Row j of $Wh+b$ is $w_j^\top h + b_j$, a linear function of h. By Lemma 2.1, its range over the box $\{h : |h - c| \leq r\}$ is exactly $[w_j^\top c + b_j - |w_j|^\top r,\ w_j^\top c + b_j + |w_j|^\top r]$. So a *single* affine layer is enclosed **tightly**. Looseness

only appears when several layers are composed.

7.3.2 Sine and cosine

sin is not monotone, so its extremes on $[l, u]$ occur at the endpoints *or* at interior critical points $\frac{\pi}{2} + k\pi$. The code (`_interval_sin`) does the following:

1. Start with $[\min(\sin l, \sin u),\ \max(\sin l, \sin u)]$.
2. Test whether $[l, u]$ contains a maximum point $\frac{\pi}{2} + 2k\pi$. With $k = \lfloor (u - \frac{\pi}{2})/2\pi \rfloor$, the largest maximum point $\le u$ is $\frac{\pi}{2} + 2k\pi$. If it is $\ge l$, set the upper bound to 1.
3. Do the same for minimum points $-\frac{\pi}{2} + 2k\pi$, setting the lower bound to -1.
4. If $u - l \ge 2\pi$, the result is $[-1, 1]$.

$\cos x = \sin(x + \frac{\pi}{2})$ reuses the same routine.

7.3.3 The derivative of tanh

$\tanh'(z) = 1 - \tanh^2(z)$ is even and decreasing in $|z|$. On $[\underline{z}, \overline{z}]$:

- the maximum is at the point closest to 0, namely $\text{clip}(0, \underline{z}, \overline{z})$;
- the minimum is at an endpoint: $\min(\tanh'(\underline{z}), \tanh'(\overline{z}))$.

Computing $1 - h \cdot h$ with interval multiplication instead would treat the two copies of h as independent and lose tightness. This is the dependency problem, next.

7.4 The dependency problem

Interval arithmetic treats every occurrence of a variable independently. For $x \in [0, 1]$:

- $x - x$ evaluates to $[0 - 1,\ 1 - 0] = [-1, 1]$, although the true value is always 0.
- $x(1 - x)$ evaluates to $[0, 1] \cdot [0, 1] = [0, 1]$, although the true range is $[0, 0.25]$.

Neural networks suffer from this badly. Neurons share inputs with positive and negative weights, and the output combines them, so correlations cancel in reality but not in the intervals.

7.4.1 Worked example: IBP on a tiny network

Consider $y = 1.5\tanh(2x - 1) + 1.0\tanh(-x + 0.5) - 0.2$ for $x \in [0, 1]$.

1. Pre-activations: $z_1 = 2x - 1 \in [-1, 1]$ and $z_2 = -x + 0.5 \in [-0.5, 0.5]$.
2. Activations: $h_1 \in [-0.7616, 0.7616]$ and $h_2 \in [-0.4621, 0.4621]$.
3. Output: $y \in 1.5[-0.7616, 0.7616] + [-0.4621, 0.4621] - 0.2 = [-1.8045,\ 1.4045]$.

The **true range**, from dense sampling, is $[-0.8803,\ 0.4803]$. IBP is more than twice too wide. The reason: h_1 increases with x while h_2 decreases, so their extremes never occur together, but interval arithmetic assumes they might.

Splitting helps. On $[0, 0.5]$ IBP gives $[-1.342, 0.262]$, and on $[0.5, 1]$ it gives $[-0.662, 0.942]$. The union is still loose, but each half is tighter. Width shrinks roughly linearly with box size. This is why branch and bound works even with loose bounds.

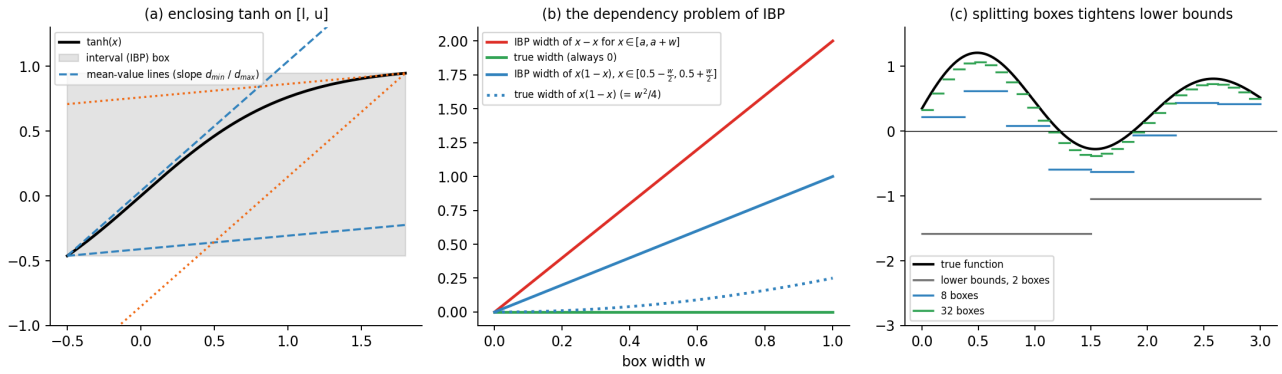


Figure 7.1: (a) Ways to enclose tanh on $[l, u]$: the interval box and the mean-value lines. (b) The dependency problem: IBP widths versus true widths. (c) Splitting into more boxes tightens lower bounds.

7.5 Floating-point soundness in the implementation

Bounds are computed in float64, and every operation widens its result outward.

```
_U = 2.0 ** -52          # unit roundoff (conservative)
_ELEM_REL = 8 * _U       # elementwise ops: widen by 8 ulp relative
_LIB_ABS = 4 * _U        # tanh/sin/cos from the math library: < 2 ulp error

def _widen(lo, hi, rel=_ELEM_REL, abs_=_ELEM_ABS):
    return lo - (rel * lo.abs() + abs_), hi + (rel * hi.abs() + abs_)
```

The affine layer uses the dot-product error bound from Chapter 2:

```
def linear(self, weight, bias=None):
    c, r = self.center(), self.radius()
    yc = c @ W.t()                          # centre
    yr = r @ W.abs().t()                    # radius
    mag = c.abs() @ W.abs().t() + yr        # sum |w||h|, magnitude of the terms
    if bias is not None:
        yc = yc + b; mag = mag + b.abs()
    n = W.shape[1] + 2
    err = 2 * n * _U * mag + _ELEM_ABS      # >= gamma_n * sum|w h|, plus centre/radius rounding
    return Interval(yc - yr - err, yc + yr + err)
```

For sin and cos, the argument itself may carry rounding error. Since $|\sin'| \leq 1$, an argument error e changes the value by at most e, and the code adds that amount.

Why float64 and not float32? float32 has $u \approx 6 \times 10^{-8}$. The widening would then be comparable to the margins we need to prove (around 10^{-2} to 10^{-3} after many operations), and CROWN's lack of directed rounding (Chapter 8) would be far riskier. The weights are trained in float32 but are **exactly representable** in float64, so the verified function is exactly the trained one.

7.6 The `Interval` class

`src/verification/interval.py` implements the operator overloads (`__add__`, `__sub__`, `__mul__`, `__neg__`, `__rsub__`, `__getitem__`) and the methods `linear`, `scale_shift`, `tanh`, `dtanh`, `sin`, `cos`, `square` and `cat`. Because `src/ops.py` dispatches on type, calling

```
out = certificate_module(Interval(lo, hi))   # lo, hi: (N, n) batches of boxes
```

runs the *same* Python code as training, but on intervals. It returns bounds on $[B, \text{cond}]$ for N boxes at

once. Batching makes IBP extremely fast: hundreds of thousands of boxes per second.

7.7 Testing soundness

`tests/test_interval_bounds.py` samples random boxes, including wide ones and ones crossing the sin extrema. It evaluates the exact function at 2000 points per box, corners included, and asserts that every value lies inside the computed interval. This is the practical meaning of "**interval lower bounds never overestimate actual function minimums**" from the project specification.

7.8 Why IBP, and why it is not enough

Strengths: trivial to implement correctly, very fast, easy to make rounding-safe. **Weakness:** the dependency problem. On the final pendulum networks, IBP alone *can* certify, but it needs **216,787** boxes, against **140** with CROWN plus Z3. That is why the next layer exists.

Chapter 8

Linear relaxations, CROWN and auto_LiRPA

8.1 Intuition

Keep, for every intermediate quantity, not a constant interval but **two linear functions of the input** that sandwich it over the box. Linear functions remember how each quantity depends on the input, so correlated quantities can cancel correctly.

8.2 Linear relaxation of a single nonlinearity

For $y = \sigma(z)$ with $z \in [l, u]$, choose slopes and intercepts with

$$a_L z + b_L \;\le\; \sigma(z) \;\le\; a_U z + b_U \qquad \forall z \in [l, u].$$

For tanh:

- **Both endpoints ≥ 0 (concave region).** The chord from $(l, \tanh l)$ to $(u, \tanh u)$ lies *below* the graph, so it is a valid lower line. Any tangent line lies *above* the graph, so it is a valid upper line.
- **Both endpoints ≤ 0 (convex region).** The situation is mirrored: the chord is above and tangents are below.
- **Mixed signs.** Use a tangent at a suitable point, chosen so that the line stays valid across the inflection point at 0.

auto_LiRPA implements these cases for tanh, sin, cos and many other functions.

8.3 Products: the McCormick envelope

The Lie derivative multiplies quantities that both depend on x: for example $\tanh'(z) \cdot dz$, $v \cos\theta$, or $\sin\theta \cdot u_1$. For $w = ab$ with $a \in [a_l, a_u]$ and $b \in [b_l, b_u]$, the four products of non-negative factors give linear inequalities:

$$\begin{aligned}
(a - a_l)(b - b_l) \ge 0 &\Rightarrow w \ge a_l b + a b_l - a_l b_l \\
(a_u - a)(b_u - b) \ge 0 &\Rightarrow w \ge a_u b + a b_u - a_u b_u \\
(a_u - a)(b - b_l) \ge 0 &\Rightarrow w \le a_u b + a b_l - a_u b_l \\
(a - a_l)(b_u - b) \ge 0 &\Rightarrow w \le a_l b + a b_u - a_l b_u
\end{aligned}$$

This is the **McCormick envelope** (McCormick, 1976): the convex hull of the graph of ab over the rectangle. It is the tightest possible *linear* relaxation of a product.

Numeric check. For $a, b \in [0,1]$ at $a = b = 0.5$ (true $w = 0.25$), the lower bounds give $w \geq 0$ and $w \geq 0.5 + 0.5 - 1 = 0$, and the upper bounds give $w \leq 0.5$ and $w \leq 0.5$. So the relaxation is $w \in [0, 0.5]$, which contains 0.25 □. It is exact at the corners of the rectangle.

8.4 CROWN: backward propagation of linear bounds

Goal. Find $\underline{A}, \underline{b}$ such that $y(x) \geq \underline{A}x + \underline{b}$ for all x in the box.

Idea (Zhang et al., 2018). Start from the output $y = W_2 h + b_2$ and replace h by its relaxation, choosing the lower or the upper line according to the **sign** of each coefficient. A positive coefficient needs the lower line of h to produce a lower bound on y; a negative coefficient needs the upper line. This yields a linear expression in z. Substitute $z = W_1 x + b_1$, which is exact. The result is linear in x:

$$y \;\geq\; \underbrace{W_2^{+}\,\mathrm{diag}(a_L)W_1 + W_2^{-}\,\mathrm{diag}(a_U)W_1}_{\underline{A}}\; x + \underline{b},$$

where $W^{+} = \max(W, 0)$ and $W^{-} = \min(W, 0)$ elementwise. Deeper networks repeat the substitution layer by layer. To choose each neuron's relaxation, CROWN needs bounds $[l, u]$ on that neuron's input, which it computes recursively with the same procedure (these are *intermediate bounds*).

Finally, **concretize** with Lemma 2.1: $\min_{x \in R} \underline{A}x + \underline{b} = \underline{A}c - |\underline{A}|r + \underline{b}$.

8.4.1 Worked example (same tiny network)

These numbers were produced by auto_LiRPA on $y = 1.5\tanh(2x - 1) + \tanh(-x + 0.5) - 0.2$:

box	IBP	CROWN	true range	CROWN lower line	CROWN upper line
$[0, 1]$	$[-1.80,\ 1.40]$	$[-1.04,\ 0.64]$	$[-0.88,\ 0.48]$	$1.500x - 1.044$	$1.500x - 0.856$
$[0, 0.5]$	$[-1.34,\ 0.26]$	$[-1.02,\ -0.19]$	$[-0.88,\ -0.20]$	$1.435x - 1.021$	$1.345x - 0.863$
$[0.5, 1]$	$[-0.66,\ 0.94]$	$[-0.21,\ 0.62]$	$[-0.20,\ 0.48]$	$1.345x - 0.882$	$1.435x - 0.814$

Observations:

- CROWN is roughly **twice as tight** as IBP on the full box.
- The linear functions track the function's *trend* (slope around 1.4–1.5), which constant bounds cannot do.
- On the half-boxes CROWN is nearly exact.

8.5 auto_LiRPA in the project

`auto_LiRPA` (Xu et al., 2020) traces any PyTorch module into a graph of "bounded operators" and runs CROWN (and other methods) automatically.

```
from auto_LiRPA import BoundedModule, BoundedTensor, PerturbationLpNorm

lirpa = BoundedModule(model_float64, torch.zeros(1, n))          # trace once
ptb = PerturbationLpNorm(norm=float("inf"), x_L=l, x_U=h)       # batch of boxes
x = BoundedTensor(0.5 * (l + h), ptb)
lb, ub, A = lirpa.compute_bounds(
    x=(x,), method="CROWN", return_A=True,
    needed_A_dict={lirpa.output_name[0]: [lirpa.input_name[0]]})
a = A[out_name][in_name]   # a["lA"], a["lbias"], a["uA"], a["ubias"]
```

- `lb`, `ub` have shape `(N, 2)`: bounds on $[B, \text{cond}]$ for each of the N boxes.
- `lA` and `uA` have shape `(N, 2, n)`: the linear bounding functions. They are needed by the Z3 step (Chapter 9) and returned at negligible extra cost.

8.5.1 Engineering details and pitfalls

- **The float64 dtype bug.** auto_LiRPA's slicing operator allocates buffers with PyTorch's *default* dtype (float32). With a float64 model this caused `index_copy_(): self and source expected to have the same dtype`. The fix (`BoundEngine.crown`) sets `torch.set_default_dtype(torch.float64)` for the duration of the call and restores it afterwards.
- **Chunking.** Boxes are processed in chunks of 2048 (`crown_batch`) to bound memory use.
- **Intersection with IBP.** Both are sound, so `max(lb_IBP, lb_CROWN)` and `min(ub_IBP, ub_CROWN)` are sound too, and at least as tight as either.
- **Rounding.** auto_LiRPA does not use directed rounding. The project requires every inequality to hold with margin $\tau = 10^{-8}$, orders of magnitude above typical float64 accumulated error (around 10^{-13}). This is standard practice in neural network verification tools, but it is not a formal guarantee about floating point. It is the main soundness caveat of the project, and it is stated in the README and the paper.
- **Graceful degradation.** If CROWN throws an exception, the verifier logs a warning and continues with IBP only (`crown_failed`).

8.6 Why CROWN (and not α-CROWN or an LP)?

- **LP relaxation** (solving an LP per bound) is tighter but orders of magnitude slower per box.
- **α-CROWN** optimises the relaxation slopes by gradient descent. It is tighter but slower. It crashed in a quick experiment on this graph, so it is listed as future work.
- **CROWN** is the sweet spot: closed-form, batched and roughly twice as tight here. The ablation (Chapter 20) shows it reduces the pendulum proof from 216,787 boxes to 165.

Chapter 9

SAT, SMT and the Z3 engines

9.1 From SAT to SMT

SAT. Given a Boolean formula such as $(p \vee \neg q) \wedge (q \vee r)$, is there an assignment of true and false to the variables that makes it true? SAT is NP-complete. Modern solvers still handle millions of variables, using **CDCL** (conflict-driven clause learning): guess a value, propagate consequences, and when a conflict occurs, learn a new clause explaining it and backtrack.

SMT (Satisfiability Modulo Theories) allows atoms from mathematical *theories* instead of plain Boolean variables:

theory	example atom	decidable?	typical method
QF_LRA: linear real arithmetic	$2x - 3y \le 1$	yes, in polynomial time for conjunctions	simplex
QF_NRA: nonlinear (polynomial) real arithmetic	$x^2 + y^2 \le 0.36$	yes, but expensive	cylindrical algebraic decomposition / nlsat
reals with sin, tanh	$\tanh(x) \ge 0.3$	**no** (in general)	only δ-complete approximations (dReal)

An SMT solver combines a SAT engine for the Boolean structure (`Or`, `And`) with theory solvers for the atoms. This architecture is called **DPLL(T)**.

9.2 Z3 basics

Z3 (de Moura and Bjørner, 2008) is Microsoft Research's SMT solver. Its Python API:

```
import z3
x = z3.Real("x")
s = z3.Solver()
s.add(x >= 0, x <= 1)
s.add(z3.RealVal("1.4999") * x + z3.RealVal("-1.0435") < 0)
print(s.check())   # sat
print(s.model())   # [x = 0]
```

- `check()` returns `sat`, `unsat` or `unknown` (for example on timeout).
- For `sat`, `model()` gives a witness.
- Reals are **exact rationals**: no rounding happens inside Z3.

9.3 Proving by refutation

To prove $\forall x \in R : \varphi(x)$, ask for $\exists x \in R : \neg\varphi(x)$:

- **UNSAT**: no counterexample exists, so φ holds on all of R. This is a proof.
- **SAT**: the model is a candidate counterexample.

The small example above uses the CROWN lower line of the tiny network, $y \geq 1.4999x - 1.0435$ on $[0, 1]$:

- Asking whether the line can drop below -1.05 returns **unsat**, which proves $y \geq -1.05$ on $[0, 1]$.
- Asking whether it can drop below 0 returns **sat** with $x = 0$. Evaluating the real network there gives $y(0) = -0.88 < 0$, so this is a *genuine* counterexample to "$y \geq 0$".

9.4 The obstacle: transcendental functions

The certificate contains tanh, sin and cos. No complete decision procedure exists for these, and Z3 does not support them natively. We therefore hand Z3 a **sound relaxation**: a formula whose set of solutions *contains* the real behaviour. The logic of refutation then works like this:

- relaxation UNSAT $\Longrightarrow$ real problem UNSAT $\Longrightarrow$ **proof** (sound);
- relaxation SAT $\Longrightarrow$ the real problem may or may not be SAT $\Longrightarrow$ the candidate must be **validated**.

The project provides two relaxations.

9.5 Engine 1 (default): pointwise reasoning over CROWN's linear bounds

For an undecided box R, CROWN has produced linear functions with

$$B(x) \leq \overline{A}_B x + \overline{b}_B, \qquad \text{cond}(x) \geq \underline{A}_c x + \underline{b}_c \qquad \forall x \in R.$$

P1 and P2 fail at a point x only if $B(x) \geq 0$ and (cond$(x) < 0$ or x is unsafe). Since $B(x) \geq 0$ implies $\overline{A}_B x + \overline{b}_B \geq 0$, and cond$(x) < 0$ implies $\underline{A}_c x + \underline{b}_c < 0$, every real violation satisfies

$$\Phi(x) := \ (\overline{A}_B x + \overline{b}_B \geq -\tau) \ \wedge \ \Big(\underline{A}_c x + \underline{b}_c < \tau \ \vee \ \bigvee\nolimits_i h_i(x) \leq 0\Big), \qquad x \in R.$$

So **UNSAT of Φ proves P1 and P2 on** R. The margins τ make the statement slightly stronger, matching the box-level tests.

Why is this stronger than CROWN's box-level test? The box test requires *either* $\max_R$(upper line of B) < 0 *or* $\min_R$(lower line of cond) ≥ 0 on the **whole** box. Z3 instead asks whether a **single point** exists where *both* bad things happen. Consider a box cut by the boundary of $\mathcal{C}$, where cond is negative only in the part where B is negative. The box test fails both alternatives. Z3 returns UNSAT because the two regions do not overlap. No splitting is needed.

Exactness choices.

- Every float coefficient of $\overline{A}, \underline{A}$ is converted to the *exact* rational it represents (`fractions.Fraction(float)`). The linear functions are then used without further rounding.
- The unsafe set is built with `Z3ExactVec`, a tiny backend in which `+`, `*` and `square` produce exact Z3 polynomial terms. For the car, $p_x^2 + p_y^2 - 0.36 \leq 0$ is encoded **exactly**. The formula is then in nonlinear real arithmetic, but it has only four variables and one quadratic atom, so Z3 solves it in milliseconds.
- Variants for the other properties: P3 (faces) asks $\exists x : \overline{A}_B x + \overline{b}_B \geq -\tau$. P0 (initial set) asks $\exists x \in X_0 : \underline{A}_B x + \underline{b}_B \leq \tau$, with the initial-set constraints $g_j(x) \geq 0$ also encoded exactly.

Code (`src/verification/smt_solver.py::z3_check_linear_bounds`, simplified):

```
xs = [z3.Real(f"x{i}") for i in range(n)]
for i in range(n):
    solver.add(xs[i] >= _exact(lo[i]), xs[i] <= _exact(hi[i]))     # x in the box
lin = lambda A, b: z3.Sum([_exact(A[i]) * xs[i] for i in range(n)]) + _exact(b)
solver.add(lin(uA[0], ubias[0]) >= _exact(-tol))                  # B may be >= 0
if mode == "domain":
    unsafe = [p.exprs[0] <= 0 for p in system.unsafe_pieces(Z3ExactVec(xs))]
    solver.add(z3.Or([lin(lA[1], lbias[1]) < _exact(tol)] + unsafe))
res = solver.check()     # unsat -> box proven; sat -> candidate
```

Validating the candidate. A SAT model is converted to floats and evaluated on the real float64 network with `is_violation(model, system, x, mode)`:

- **genuine** if $B(x) \geq 0$ and ($\mathrm{cond}(x) < 0$ or x is unsafe). It is recorded as a counterexample and later used for retraining.
- **spurious** otherwise: the relaxation was too loose, so the box is split.

Measured effect. On the pendulum, Z3 closed 11 of the 140 final proof boxes. In the verifier ablation, the proof shrinks from 165 boxes (CROWN only) to 140. On the Dubins car, 2,949 leaves were closed by Z3. Each call takes a few milliseconds.

9.6 Engine 2: full-graph relaxation (`z3_check_box`)

Engine 2 encodes the **entire** closed-loop computation in QF_LRA, using the `Z3RelaxedVec` backend of `ops`. Every intermediate quantity becomes a Z3 variable with interval bounds, related to its inputs by these constraints:

operation	encoding
affine $y = Wx + b$	linear equality (two inequalities)
smooth scalar $y = f(x)$, $x \in [l, u]$ (tanh, tanh′, sin, cos)	the four mean-value inequalities of Chapter 2, with $[d_{\min}, d_{\max}]$ an interval enclosure of f' on $[l, u]$
product $w = ab$	McCormick envelope
square	McCormick, plus $w \geq$ the interval lower bound

The derivative bounds used are: for tanh, the interval of tanh′; for tanh′, the interval of tanh″ $= -2\tanh \cdot \tanh'$; for sin, that of cos; for cos, that of $-\sin$. All are computed with the `Interval` class.

9.6.1 Why engine 2 is not the default

In the first version every float coefficient was converted to its exact rational. Floats have 53-bit mantissas, so these rationals have denominators up to 2^{52} and beyond. Z3's simplex manipulates exact rationals, and during pivoting their numerators and denominators grow. On a 2×32 network (about 500 variables, 2000 constraints), `check()` did not return within minutes and ignored its timeout.

The second attempt rounded all coefficients to a dyadic grid 2^{-K}. To keep the encoding sound, each constraint $\sum a_i v_i + c \geq 0$ is replaced by

$$\sum \tilde{a}_i v_i + \tilde{c} + \text{slack} \geq 0, \qquad \text{slack} \;\geq\; \sum_i |a_i - \tilde{a}_i| \max|v_i| + |c - \tilde{c}|.$$

This is valid because every variable has known bounds $|v_i| \leq \max|v_i|$. It is implemented in `Z3Context.geq0`, which also adds $10^{-13}|a|$ per term for float errors made while forming the coefficients. With $K = 10$ the solver was fast but the loosening was far too large. With $K = 16$–30 it timed out again.

Conclusion. For networks of this size, a full LRA encoding is the wrong tool. Engine 1 keeps Z3's strengths (exactness, disjunctions, polynomial sets) and leaves the heavy lifting to CROWN. Engine 2 is kept because it is correct, demonstrates the fully symbolic approach, and works on small networks. See `test_full_graph_relaxation_encoding_small_net`, which uses a 4-neuron barrier.

9.7 The spec-level helper

`extract_counterexample_z3(cert, system, box_min, box_max, mode)` computes CROWN bounds for one box, runs engine 1, validates the model, and returns the counterexample or `None`. It matches the interface described in the original project plan. `test_z3_finds_genuine_boundary_counterexample` and `test_z3_unsafe_set_is_encoded_exactly` exercise it.

Chapter 10

Branch and bound

10.1 The algorithm

Branch and bound (B&B) turns "bounds are loose on big boxes" into a complete procedure. It maintains a queue of undecided boxes, bounds each one, and splits the undecided ones. In pseudocode:

```
queue <- [D]
while queue not empty and budget left:
    take a batch of boxes from the front of the queue
    for each box R in the batch:
        if the real network violates the property at centre(R) or at a random point of R:
            record counterexample; if R is large, split it anyway (exploration); continue
        compute IBP bounds;   if they prove R: record leaf (IBP); continue
        compute CROWN bounds; if they prove R: record leaf (CROWN); continue
        if R is small enough for Z3:
            Z3 UNSAT            -> record leaf (Z3); continue
            Z3 SAT and genuine  -> record counterexample; continue
        if R is at the resolution limit: mark R unknown
        else: bisect R along its widest (domain-normalised) coordinate; append both halves
```

10.2 The per-box tests

With $\overline{B}$ an upper bound of B and $\underline{\text{cond}}$ a lower bound of cond over the box:

mode	region	proven if
`domain` (P1 + P2)	D	$\overline{B} < -\tau$ (the whole box is outside $\mathcal{C}$), **or** ($\underline{\text{cond}} \geq \tau$ **and** the box provably misses X_{unsafe})
`boundary` (P3)	each non-periodic face of D	$\overline{B} < -\tau$
`init` (P0)	the box enclosing X_0	$\underline{B} > \tau$, **or** the box provably misses X_0

"Provably misses the unsafe set" is decided with intervals: the box misses $\{h_i \leq 0\}$ if the interval lower bound of h_i on the box is > 0 (`BoundEngine.unsafe_may_intersect`). For the initial set, the box misses X_0 if some g_j has interval upper bound < 0 (`outside_init`). Boxes intersecting the unsafe set are allowed only if $\overline{B} < -\tau$ there.

Faces are lower-dimensional boxes. For the face $p_x = 2$, the region has $l_1 = u_1 = 2$. The bisection step never splits a zero-width coordinate, because its relative width is 0, never the widest.

10.3 Correctness

Theorem 10.1 (soundness of B&B). If the algorithm terminates with every leaf proven, no counterexamples and no unknown boxes, the property holds on the whole region.

Proof. Bisection produces a partition: every point of the original region lies in at least one leaf. Leaves share only boundary faces, which does not matter for a "for all points" statement. Each leaf was proven by a sound method (Chapters 7–9). Hence every point satisfies the property. □

Termination. Boxes are never split below `max_depth` in every coordinate, and `max_boxes` caps total work. The algorithm always terminates, possibly with UNKNOWN.

10.4 Complexity: the curse of dimensionality

Suppose a region needs boxes of relative width ε to be proven. A uniform grid then needs ε^{-n} boxes in n dimensions:

n	$\varepsilon = 2^{-5}$	$\varepsilon = 2^{-7}$	$\varepsilon = 2^{-10}$
2	1,024	16,384	10^6
4	10^6	2.7×10^8	10^{12}

B&B is *adaptive*: it refines only where needed, which is why the pendulum proof has just 140 leaves. In 4-D, however, even adaptive refinement needs millions of boxes near the difficult boundary. This is the main scalability limit (Chapter 22).

10.5 Implementation details (`BranchAndBoundVerifier.run`)

- **Breadth-first order** (a `collections.deque`) refines the whole region evenly, so a budget-limited run leaves "uniformly unfinished" work instead of one fully refined corner.
- **Batching.** Boxes are processed in batches of 8192 (pendulum) or 16384 (car). IBP and CROWN are vectorised over the batch, which gives a large speed-up on CPU.
- **Early stop on counterexamples.** After `max_cex` (512) counterexamples, the verifier stops. During synthesis there is no point proving the rest of a network that is already known to be wrong.
- **Exploration of falsified boxes.** A falsified large box (relative width above $2^{-\texttt{cex_split_depth}}$) is still split, so the rest of its volume is examined and more distinct counterexamples are harvested. An earlier version dropped falsified boxes, and verification of the car then stopped at the root box: its centre (the origin) lies inside the obstacle.
- **Z3 gating.** Z3 is only tried on boxes smaller than $2^{-\texttt{z3_depth}}$, where CROWN's linear bounds are accurate enough for pointwise reasoning to succeed, up to `z3_max_boxes` calls per run.
- **Relative widths.** Coordinates have different units (radians, metres, m/s), so "widest" is measured relative to the domain width in that coordinate.
- **Alternative branching** (`smart_split`). Split the coordinate d maximising $|A_d| \cdot \text{width}_d$, that is, the coordinate contributing most to the linear bound's range. This was implemented and measured: on the car checkpoint, with the same 150k-box budget, it proved **63.9%** of the volume against **89.2%** for the widest-coordinate rule. It is therefore off by default.
- **Result objects.** `BBResult` stores leaves, methods, whether each leaf is outside $\mathcal{C}$, counterexamples, unknown boxes and statistics. `VerificationResult` aggregates the domain, all faces and the initial set, computes `status`, and reports `proven_volume_fraction`: the fraction of $\text{vol}(D)$ covered by proven leaves.

Chapter 11

The certificate and the proof checker

11.1 What is stored

`save_certificate` (in `cgis_loop.py`) writes two files.

`certificate.pt` (a PyTorch pickle, gzip-compressed for the car) contains:

- `controller`, `barrier`: the network weights (`state_dict`);
- `gamma` and `config`: everything needed to rebuild the system and the networks;
- `verification`: for the domain, each face and the initial set, the leaf boxes (`leaves_lo`, `leaves_hi`), the proving method of each leaf (0 = IBP, 1 = CROWN, 2 = Z3), the outside-$\mathcal{C}$ flags, counterexamples and unknown boxes;
- `history`: one summary per synthesis round.

`certificate.json` is a human-readable summary. It includes the **SHA-256 hashes** of the controller and barrier weights and the list of certified properties in words. Anyone can confirm that the networks they deploy are byte-for-byte the certified ones.

11.2 The trusted computing base

A proof is only as trustworthy as the code that checks it. The **trusted computing base** (TCB) here consists of:

- the system definitions and network forward code;
- the interval arithmetic;
- auto_LiRPA's CROWN;
- Z3;
- the checker itself.

The training code, the loss functions and the B&B search heuristics are **not** in the TCB. The checker ignores them completely.

11.3 What the checker does (`ProofChecker.check`)

1. **Rebuild** the networks from the stored weights via `load_certificate`.
2. **Coverage check** for each region (domain, each face, initial box):
   - every leaf lies inside the region;
   - the total leaf volume equals the region volume, to relative tolerance 10^{-9} (for faces, volume is measured in the non-degenerate coordinates).

   Because the leaves come from bisection they have disjoint interiors, so *equal volume implies the leaves cover the region* (up to measure-zero boundaries).
3. **Re-proof.** Every leaf is re-proven with freshly computed bounds, using its recorded method: IBP

leaves with IBP; CROWN leaves with IBP ∩ CROWN; Z3 leaves with a new CROWN computation and a new Z3 call.

4. **Accept** only if coverage holds everywhere, no leaf fails, and there are no recorded counterexamples or unknown boxes.

Evidence that it works.

- The pendulum certificate is accepted in 1.5 s.
- `test_proof_checker_rejects_tampered_certificate` adds 3.0 to one controller bias, and the checker rejects the certificate.
- During development, a version of the synthesis loop trained once more *after* the last verification. The checker rejected the resulting certificate because the weights no longer matched the proof. That was a real bug, caught exactly as intended (Chapter 21).

Part IV — Synthesis

Chapter 12

Learning the controller and the barrier

12.1 The learning problem

We want weights (θ, ϕ) such that P0–P3 hold. Training cannot enforce "for all x". It can only penalise violations at **sampled** states. Training therefore produces *candidates*, and the verifier judges them.

12.2 Hinge losses with margins

Each property "$q(x) \geq 0$" becomes the hinge loss $\mathrm{ReLU}(m - q(x))$. It is zero when $q \geq m$ and grows linearly with the violation. The margin $m > 0$ asks for **slack** beyond the verifier's requirement:

- the verifier needs strict inequalities, with margin τ;
- bounds are loose, so a property that holds with value 10^{-6} is practically unprovable, while one that holds with value 10^{-2} is provable on moderately sized boxes.

Margins used: $m = 0.02$ (pendulum) and $m = 0.01$ (car).

12.3 The loss terms (`CGISTrainer.losses`)

term	formula	samples	enforces
`lie`	$\sum \mathrm{ReLU}(m - \mathrm{cond}(x))\, w(x) \;/\; \sum w(x)$	domain (mixed, see below)	P1
`unsafe`	mean $\mathrm{ReLU}(B(x) + m)$	unsafe set (inflated for the car)	P2
`boundary`	mean $\mathrm{ReLU}(B(x) + m)$	faces of D	P3
`init`	mean $\mathrm{ReLU}(m - B(x))$	X_0	P0
`cgis`	same hinges on buffered counterexamples	CGIS buffer	all
`lip`	mean of $\left(\frac{\|\pi(x+d) - \pi(x)\|/\mathrm{span}}{\|d\|/\mathrm{width}}\right)^2$	domain	smooth controller
`nominal`	mean $\|(\pi(x) - u_{\mathrm{nom}}(x))/\mathrm{span}\|^2$	domain	task (car only)

The total loss is the weighted sum $w_{\mathrm{lie}}\mathcal{L}_{\mathrm{lie}} + w_{\mathrm{unsafe}}\mathcal{L}_{\mathrm{unsafe}} + \ldots$. The CGIS term implements the formula from the project plan, $\mathcal{L}_{\mathrm{CGIS}} = \max\left(0, -(\dot{B} + \gamma B)\right)$ at counterexamples, with the margin added.

12.4 The smooth band: why the Lie term uses a sigmoid

P1 matters only where $B \geq 0$. The obvious loss uses a **hard mask** $\mathbb{1}[B(x) \geq -\delta]$. The mask is piecewise constant in B, so its gradient with respect to ϕ (the barrier's weights) is zero. The optimiser can then reduce a violation at a state x only by **increasing** $\text{cond}(x)$. At states where no admissible input can do that (a pendulum beyond the torque limit, falling), the loss pushes forever without success. The observed symptom was thin "arms" of $\mathcal{C}$ extending into infeasible regions. Those arms made verification impossible.

The fix is a smooth weight $w(x) = \sigma\big((B(x) + \delta)/T\big)$, with $\delta = 0.01$ and temperature $T = 0.01$. Now

$$\frac{\partial}{\partial B}\Big[\text{ReLU}(m - \text{cond})\, w\Big] \ni \text{ReLU}(m - \text{cond})\, \frac{\sigma'(\cdot)}{T} \; > 0$$

wherever the condition is violated. Gradient descent therefore also **lowers** B at such states, *shrinking* $\mathcal{C}$ to exclude them. This single change turned a non-converging loop into a pendulum certificate within 3 rounds.

A small band matters too. An earlier version used $\delta = 0.2$ on a barrier whose values lay mostly in $[-0.9, 0.1]$. That required cond $\geq m$ on a large region *outside* $\mathcal{C}$, including falling states where it is physically impossible. Training stalled.

12.5 Sampling strategy

Each training step draws fresh samples:

- **domain batch** (2048 or 4096 states). Half are uniform in D. The other half come from the band $B \geq -\delta$, found by rejection sampling from 4× as many candidates (`_focused_domain_batch`). P1 is decided in the band, so that is where samples help most;
- 1024 samples each from the unsafe set, the faces and X_0 (rejection sampling for the non-box sets);
- up to 1024 domain counterexamples, plus 256 face and 256 initial-set counterexamples, from the CGIS buffers.

Training-only inflation of the obstacle (car, `train_inflation=0.08`). Unsafe samples are drawn from a disc of radius $0.6 + 0.08$. This pushes $\mathcal{C}$ away from the obstacle surface, where the condition is hardest (Chapter 20). Verification always uses the true radius 0.6.

12.6 The controller smoothness penalty

Without regularisation, the pendulum controller became **bang-bang**: u jumped from $+6$ to -6 across a thin surface. Over a box only 0.002 wide the controller output varied by 0.15, so interval bounds on everything downstream were hopeless. The finite-difference penalty

$$\mathcal{L}_{\text{lip}} = \mathbb{E}\left[\left(\frac{\|\pi(x+d) - \pi(x)\| / (u_{\max} - u_{\min})}{\|d / (\text{domain width})\|}\right)^2\right], \qquad d \sim \mathcal{N}(0, (0.01 \cdot \text{width})^2),$$

with weight 10^{-3}, keeps the controller smooth enough to verify at little cost in performance.

An optional **verification-friendly** penalty on the local slopes of $[B, \text{cond}]$ (`w_cert_lip`) is also implemented. It is off by default, because diagnostics showed the car's CROWN looseness came from the relaxation itself, not from steep functions.

12.7 The nominal controller (car)

To make the car do something useful, a weak imitation term pulls π towards a goal-seeking controller:

$$\theta_{\text{des}} = \text{atan2}(g_y - p_y,\ g_x - p_x), \quad u_1^{\text{nom}} = \text{clip}(2 \cdot \text{wrap}(\theta_{\text{des}} - \theta)), \quad u_2^{\text{nom}} = \text{clip}(2(v_{\text{des}} - v)).$$

Its weight (0.02, later 0.005) is small, so safety terms dominate wherever they conflict.

12.8 Optimisation

- **Adam** with $\beta = (0.9, 0.999)$ and weight decay 10^{-5}. Adam keeps running averages of the gradient (m_t) and of its square (v_t), and updates with $\theta \leftarrow \theta - \eta\, \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$, which adapts the step size per parameter.
- **Cosine learning-rate decay** in each training phase: $\eta_t = \eta_{\min} + \frac{\eta_0 - \eta_{\min}}{2}(1 + \cos\frac{\pi t}{T})$, with $\eta_{\min} = 0.05\,\eta_0$. With a constant rate, violations kept fluctuating around small non-zero values. Decay lets them settle.
- Learning rates: 10^{-3} (pendulum) and 3×10^{-3} (car). Initial phase: 2000 steps (pendulum) or 10000 (car). Each later round: 500 or 1500 steps.
- **Loss weights.** $w_{\text{init}} = 5$ (pendulum) and 10 (car), because P0 otherwise loses to the counterexample terms, which push B down; $w_{\text{unsafe}} = w_{\text{boundary}} = 10$ for the car; $w_{\text{cex}} = 2$ (pendulum), 5 (car).

Chapter 13

Counterexample-guided inductive synthesis (CGIS)

13.1 Origin of the idea

CEGIS (*counterexample-guided inductive synthesis*) comes from program synthesis (Solar-Lezama et al., 2006). A **learner** generalises from finitely many examples. A **verifier** checks the candidate against the full specification and returns a counterexample when it fails. The counterexample becomes a new example. Neural Lyapunov and barrier synthesis (Chang et al. 2019; Abate et al. 2021) adopted it with SMT verifiers. This project uses a branch-and-bound verifier with Z3 for the hard boxes.

13.2 The loop (`CGISTrainer.run`)

```
train(steps_initial)
for round r = 0, 1, ...:
    res = verify(quick)                      # coarse falsification pass (if configured)
    if quick and no counterexamples:
        res = verify(full)                   # the real proof attempt
    log the round summary
    if res is CERTIFIED (full), or r is the last round: stop
    add counterexamples (+ jittered copies) and centres of unknown boxes to the buffers
    train(steps_per_round)
    save checkpoint
save certificate; run the independent proof checker
```

13.3 Design decisions

- **Counterexample neighbourhoods.** Each counterexample is copied 16 times with Gaussian jitter (2% of the domain width). Face counterexamples keep their fixed coordinate on the face. Initial-set copies are filtered to stay inside X_0. Training on the neighbourhood removes the violation region, not just one point.
- **Unknown boxes as hard examples.** The centres of up to 2048 unknown boxes are added too. They mark regions where the margin is thin.
- **Buffer cap.** At most 200,000 points, first-in-first-out.
- **Quick vs full verification.** For the car, one full verification took around 15 minutes or more. Intermediate rounds therefore run a quick pass (maximum depth 7, 150k boxes) whose purpose is to find counterexamples. Only a quick pass with **zero** counterexamples triggers the full proof. A quick pass can never certify on its own.

- **Checkpointing and resume.** After every round, the weights, optimiser state, buffers and history go to `checkpoint.pt`. `resume=true` continues from it. This saved the Dubins run after an out-of-memory kill.
- **Saved weights equal verified weights.** The loop stops *immediately after* the last verification. An earlier version trained once more first, so the saved networks differed from the verified ones and the proof checker rejected them.

13.4 Convergence

CGIS has **no convergence guarantee**. The learner may keep producing new failures, or the network class may be unable to satisfy the conditions with the required margins. In this project:

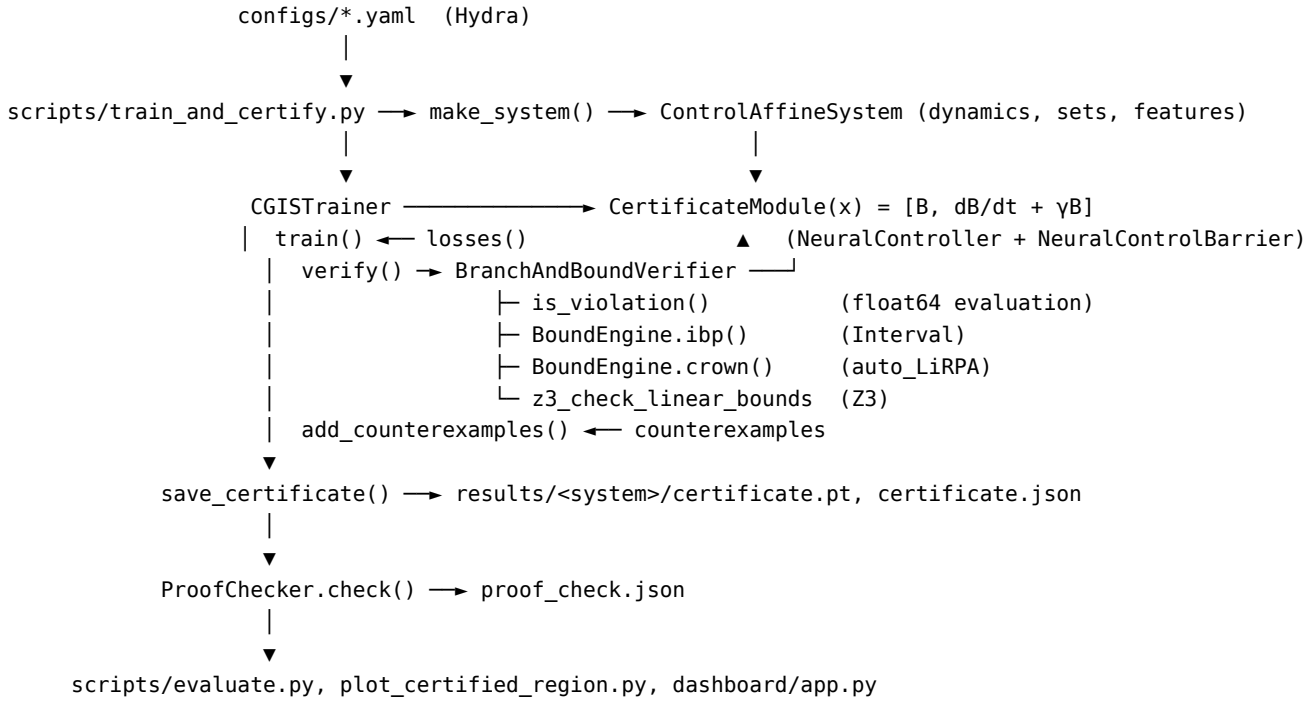
- **Pendulum:** 72 → 7 → 0 counterexamples, certified in round 2 (3 rounds counting from 0).
- **Car:** counterexamples fell to about 5 per quick round, but never reached zero within 40 rounds (Chapter 20).

Part V — Implementation

Chapter 14

Architecture and data flow

14.1 Component diagram



14.2 The “one function, many backends” principle

Only one function is ever verified: `CertificateModule.forward`. It is called with four kinds of input:

call	backend	produced by
`cert(x_tensor)`	exact values; differentiable	training, falsification, simulation
`cert(Interval(lo, hi))`	interval enclosures	`BoundEngine.ibp`
`BoundedModule(cert)`	auto_LiRPA traces the tensor path	`BoundEngine.crown`
`cert(Z3RelaxedVec(...))`	Z3 constraints	`z3_check_box` (engine 2)

Chapter 15

Source code reference

This chapter lists every module and function, with its purpose, inputs and outputs.

15.1 `src/ops.py`: backend dispatch

function	tensor behaviour	other backends
`sin(x)`, `cos(x)`, `tanh(x)`	`torch.sin` etc.	`x.sin()` etc.
`dtanh(z, h)`	`1 - h*h`	`z.dtanh()` (uses the pre-activation for tightness)
`linear(x, W, b)`	`F.linear`	`x.linear(W, b)`
`scale_shift(x, a, c)`	`x*a + c`	`x.scale_shift(a, c)`
`cat(xs)`	`torch.cat(dim=1)`	`type(x).cat(xs)`

15.2 `src/dynamics/continuous_system.py`: systems

`Box(lo, hi)` is a dataclass for axis-aligned boxes; `.tensor()` returns `(lo, hi)` tensors.

`ControlAffineSystem` is the base class. Every system has a `domain`, `u_min`/`u_max`, an `init_set` (bounding box of X_0) and a list of `periodic` coordinates.

method	purpose
`state_dim`, `control_dim`, `feature_dim`	sizes
`xdot(x, u)`	**abstract**: closed-loop vector field $f + gu$ (backend-agnostic)
`features(x)` / `features_jvp(x, xdot)`	**abstract**: $\psi(x)$ and $J_\psi(x)\,\dot{x}$
`unsafe_pieces(x)`	**abstract**: list of $h_i(x)$; the unsafe set is $\bigcup\{h_i \le 0\}$
`init_constraints(x)`	list of $g_j(x)$; $X_0 =$ `init_set` $\cap\{g_j \ge 0\}$ (default: none)
`f(x)`, `g(x)`	derived drift and input matrix
`is_unsafe`, `in_domain`, `in_init`	membership tests on tensors (periodic coordinates are ignored by `in_domain`)
`sample_domain`, `sample_box`, `sample_init`, `sample_unsafe`, `sample_boundary`	samplers (rejection sampling for non-box sets)
`sample_unsafe_train`	unsafe samples for the loss (possibly inflated)
`boundary_faces()`	list of `(dim, side, value)` for non-periodic faces
`rhs_numpy(x, u)`	NumPy wrapper for integrators
`nominal_control(x)`	optional task controller (default: `None`)

`InvertedPendulum(m, l, b, grav, u_max, theta_unsafe, theta_lim, omega_lim, init_theta, init_omega, omega_scale)` and **`DubinsCar(arena, v_max, omega_max, a_max, obstacle_center, obstacle_radius, init_xy,`**

`init_v, goal, v_des, train_inflation)` implement the abstract methods (Chapter 3).

`make_system(cfg)` builds a system from a configuration dictionary with a `name` key. **`_square(x)`** squares tensors and dispatches `.square()` for the other backends.

15.3 `src/models/`: networks

- **`neural_barrier.py`**
  - `build_tanh_mlp(in_dim, hidden, out_dim)` returns a `ModuleList` of `nn.Linear`.
  - `mlp_forward(layers, h, dh=None)` runs the backend-agnostic forward pass, optionally with the tangent.
  - `NeuralControlBarrier(feature_dim, hidden)` has `forward(s)`, `forward_with_tangent(s, ds)` and `compute_lie_derivative(s, u, f_x, g_x)` (the reverse-mode reference from the specification, used in tests).
- **`neural_controller.py`**: `NeuralController(feature_dim, u_min, u_max, hidden)` stores the buffers `u_mid` and `u_half` and implements `forward(s)`.
- **`certificate.py`**
  - `CertificateModule(system, controller, barrier, gamma)` has `terms(x)` → `(B, dB, u)`, `forward(x)` → `[B, cond]`, and the helpers `barrier_value(x)` and `control(x)`.
  - `build_models(system, ctrl_hidden, barrier_hidden)` constructs both networks.

15.4 `src/verification/interval.py`

- **`Interval(lo, hi)`**: batched intervals of shape `(N, d)`.
  - Arithmetic: `+ - * neg`, with float and tensor constants.
  - `square`, `linear`, `scale_shift`, `tanh`, `dtanh`, `sin`, `cos`.
  - `cat`, `point`, `center`, `radius`, `contains`, `__getitem__`.
- **`_widen(lo, hi)`**: outward rounding.
- **`_interval_sin(lo, hi, shift_err)`**: the sine enclosure of Chapter 7.

15.5 `src/verification/lirpa_verifier.py`

- **`verify_state_box(model, x_center, eps)`** is the specification's helper: CROWN on an L_∞ ball, returning True if every lower bound is ≥ 0.
- **`BoundEngine(cert, use_crown)`** keeps a float64 copy of the certificate and provides:
  - `ibp(lo, hi)`;
  - `crown(lo, hi)`, which returns `{lb, ub, lA, lbias, uA, ubias}`;
  - `unsafe_may_intersect(lo, hi)`;
  - `outside_init(lo, hi)`.
- **`BBResult`** holds the result of one region: leaves, methods, outside flags, counterexamples, unknown boxes, Z3 statistics, time, and whether the budget was exhausted. Its `certified` property is true when there are no counterexamples, no unknown boxes and the budget was not exhausted.
- **`VerificationResult`** aggregates `domain`, `faces` and `init` and provides `status`, `counterexamples()`, `unknown_boxes()`, `proven_volume_fraction()` and `summary()`.
- **`BranchAndBoundVerifier(cert, use_crown, use_z3, tol, batch_size, max_boxes, max_depth, max_cex, z3_max_boxes, z3_timeout_ms, z3_depth, cex_split_depth, smart_split, verbose)`** provides:
  - `run(lo, hi, mode)`: branch and bound over one region;
  - `verify()`: runs the initial set, the domain and every face;
  - `_passes`: the box tests;
  - `_split`: bisection;
  - `_record`: stores leaves.
- **`METHOD_CODES`**: `{"ibp": 0, "crown": 1, "z3": 2}`.

15.6 src/verification/smt_solver.py

- **Engine 2** (full-graph relaxation):
  - `_qround` rounds to the dyadic grid.
  - `Z3Context` provides `new_var`, `geq0` and `leq0` (soundly loosened constraints).
  - `Aff` is an affine form $\sum a_i v_i + c$.
  - `Z3RelaxedVec` is the backend class, with McCormick products and `_c1` mean-value relaxations.
  - `z3_check_box(cert, system, lo, hi, mode, tol, timeout_ms)` returns a `Z3BoxResult`.
- **Engine 1** (CROWN linear bounds):
  - `Z3ExactVec` builds exact polynomial expressions.
  - `_exact(float)` converts a float to its exact rational.
  - `z3_check_linear_bounds(system, lo, hi, lA, lbias, uA, ubias, mode, tol, timeout_ms)`.
- **Shared:**
  - `Z3BoxResult(status, candidate, genuine, seconds)`.
  - `_model_float` converts a Z3 value to a float, handling algebraic numbers.
  - `is_violation(cert, system, x, mode)` checks points exactly in float64.
  - `extract_counterexample_z3(...)` is the specification-level helper.

15.7 src/verification/cgis_loop.py

- **`TrainConfig`** and **`VerifyConfig`** are dataclasses with every hyperparameter (Chapter 16).
- **`weights_sha256(module)`** hashes the weights.
- **`CGISTrainer(system, gamma, train_cfg, verify_cfg, ctrl_hidden, barrier_hidden, seed)`** provides:
  - `losses`, `train` and `_focused_domain_batch` (training);
  - `_expand` and `add_counterexamples` (the counterexample buffers);
  - `verify(quick)` and `run(max_rounds, verbose, checkpoint, resume)` (the loop);
  - `save_checkpoint`, `load_checkpoint` and `save` (persistence).
- **`save_certificate(out, cert, res, cfg, history)`** writes `certificate.pt` and `certificate.json`.

15.8 src/verification/proof_checker.py

- **`load_certificate(path)`** reads `.pt` or `.pt.gz` and rebuilds the system and networks.
- **`CheckReport(ok, details, seconds)`** holds the checker's verdict.
- **`ProofChecker(cert, tol)`** provides `check(verification)`, plus `_coverage`, `_passes`, `_recheck_leaves` and `_check_part`.

15.9 src/visualization/phase_portrait.py

- `simulate(cert, x0, T)`: SciPy RK45 rollout.
- `_grid_eval`: evaluates $[B, \text{cond}]$ on a 2-D grid slice.
- `draw_leaves`: draws proof boxes coloured by method, and outlines leaves that lie outside $\mathcal{C}$.
- `plot_pendulum`, `plot_dubins_slice`, `plot_dubins_trajectories`: the figures.
- `_sample_in_C`: random states with $B \geq 0$.

15.10 scripts/

script	what it does
`train_and_certify.py`	Hydra entry point: builds everything, runs CGIS, saves, runs the proof checker
`reverify.py`	re-verifies stored networks with new budgets, rewrites the certificate and the check report

script	what it does
`check_certificate.py`	runs the independent checker; exit code 0 if accepted
`evaluate.py`	verifier ablation, γ sweep, 2 M-sample check, rollouts, rollouts from counterexamples → `evaluation.json`
`plot_certified_region.py`	figures for a system
`make_paper_macros.py`	writes the Dubins results into `paper/results_macros.tex`
`make_guide_figures.py`	the teaching figures in `docs/figures`

Chapter 16

Configuration reference

The configuration is composed by Hydra from `configs/config.yaml` (defaults) and `configs/system/<name>.yaml` (per-system overrides, marked `# @package _global_`). Any key can be overridden on the command line, for example `python scripts/train_and_certify.py system=dubins_car train.margin=0.02`.

16.1 Top level

key	default	meaning
`seed`	0	random seed
`gamma`	1.0	class-$\mathcal{K}$ slope γ
`max_rounds`	20 (40 for the car)	synthesis rounds
`output_dir`	`results/${system.name}`	where results go
`resume`	false	continue from `checkpoint.pt`
`model.ctrl_hidden`, `model.barrier_hidden`	[32, 32]	hidden widths

16.2 `train.*`

key	pendulum	car	meaning
`lr`	1e-3	3e-3	Adam learning rate
`steps_initial` / `steps_per_round`	2000 / 500	10000 / 1500	training steps
`batch_domain`	2048	4096	domain samples per step
`batch_unsafe`, `batch_boundary`, `batch_init`, `batch_cex`	1024 each	1024 each	other sample sizes
`margin`	0.02	0.01	hinge margin m
`lie_band`, `lie_temp`	0.01, 0.01	same	band δ and temperature T
`w_lie`, `w_unsafe`, `w_boundary`, `w_init`, `w_cex`	1, 1, 1, 5, 2	1, 10, 10, 10, 5	loss weights
`w_nominal`	0	0.005	imitation weight
`w_lipschitz`, `lip_delta`	1e-3, 0.01	same	controller smoothness
`w_cert_lip`, `cert_lip_target`	0, 1.0	same	optional certificate-slope penalty
`focus_fraction`	0.5	same	share of band samples
`lr_final_ratio`	0.05	same	cosine decay floor
`weight_decay`	1e-5	same	L2 regularisation
`cex_jitter`, `cex_copies`, `max_buffer`	0.02, 16, 200000	same	counterexample buffer

16.3 `verify.*`

key	pendulum	car	meaning
`use_crown`, `use_z3`	true	true	enable engines
`tol`	1e-8	1e-8	margin τ
`batch_size`	8192	16384	boxes per batch
`max_boxes`	2,000,000	6,000,000	budget per region
`max_depth`	12	10	finest box = width × $2^{-\text{depth}}$
`max_cex`	512	512	early stop
`z3_depth`, `z3_max_boxes`, `z3_timeout_ms`	4, 20000, 5000	6, 5000, 5000	Z3 gating
`cex_split_depth`	4	4	exploration of falsified boxes
`smart_split`	false	false	alternative branching
`quick_max_depth`, `quick_max_boxes`	none	7, 150000	quick passes during synthesis

16.4 System parameters

These were listed in Chapter 3. The car additionally has `train_inflation: 0.08`.

Chapter 17

Running the project

17.1 Installation

```
conda create -n certicontrol python=3.11 -y
conda activate certicontrol
pip install -r requirements.txt
```

- **Python 3.11** is required by current auto_LiRPA (`requires-python ~=3.11`).
- **PyTorch.** auto_LiRPA pins `torch<2.12`. The project was also tested with 2.14.
- **auto_LiRPA** is installed from GitHub. The PyPI package is an old version (0.3).
- **CPU only.** Verification needs float64, which Apple MPS lacks.

17.2 Step-by-step tutorial

1. **Synthesise and certify the pendulum** (2 to 10 minutes): `python scripts/train_and_certify.py`. Each round prints loss lines such as `step 499 lie=0.0016 unsafe=0.0050 ...`, then a JSON summary. The run ends with `[result] CERTIFIED` and `[proof checker] ACCEPTED`.
2. **Check the proof independently:** `python scripts/check_certificate.py results/inverted_pendulum/certificate.pt` prints per-region details and `CERTIFICATE ACCEPTED`.
3. **Evaluate:** `python scripts/evaluate.py --system inverted_pendulum` writes `evaluation.json` (ablation, γ sweep, sampling, rollouts).
4. **Plot:** `python scripts/plot_certified_region.py --system inverted_pendulum`.
5. **Explore interactively:** `streamlit run dashboard/app.py`.
6. **Run the tests:** `pytest` (27 passed and 1 skipped, in about 20 s).
7. **Car** (hours, about 4 GB RAM): `python scripts/train_and_certify.py system=dubins_car`. Resume with `resume=true`. Re-verify with a larger budget with `python scripts/reverify.py --system dubins_car --max-boxes 3000000`.

17.3 Reading a round summary

field	meaning
`status`	CERTIFIED / FALSIFIED / UNKNOWN
`domain_boxes_processed`	boxes examined in D
`domain_leaves`	proven boxes in D
`proven_volume_fraction`	share of vol(D) proven
`leaves_by_method`	how many leaves each engine closed
`leaves_outside_safe_set`	leaves proven because $\overline{B} < 0$
`face_leaves`, `init_leaves`	proof sizes for P3 and P0

field	meaning
`init_set_certified`	whether P0 holds
`z3_calls`, `z3_proved`, `z3_spurious`	Z3 activity
`counterexamples`, `unknown_boxes`	failures
`full_verification`	false for quick passes

17.4 Troubleshooting

symptom	cause	fix
`index_copy_ ... Float and Double`	auto_LiRPA with float32 default dtype	already handled in `BoundEngine.crown`; do not call auto_LiRPA directly on float64 models without setting the default dtype
process killed during car verification	out of memory (the run uses about 4 GB; two heavy jobs together exceed 6 GB)	run one job at a time; resume with `resume=true`
certificate collapses ($B < 0$ everywhere)	P0 weight too small	raise `train.w_init`
many UNKNOWN boxes	budget too small, or margins too thin	raise `verify.max_boxes` / `max_depth`, or `train.margin`
Z3 hangs	using engine 2 on large networks	use engine 1 (default)

Chapter 18

Tests and the dashboard

18.1 The test suite (`pytest`)

`tests/test_interval_bounds.py` covers bound soundness and differentiation:

- `test_elementwise_ops_are_sound[tanh|sin|cos|dtanh|square]` checks 400 random boxes (widths up to 8) with 2000 samples each: every value lies inside the interval.
- `test_sin_attains_extrema` checks that intervals containing $\pm\frac{\pi}{2}$ reach ± 1.
- `test_binary_and_linear_ops_are_sound` covers `* + -`, constant scaling, `linear` and `scale_shift`.
- `test_point_interval_contains_exact_value`: degenerate intervals contain the exact value, with width below 10^{-9}.
- `test_closed_loop_ibp_is_sound[pendulum|car]`: IBP of the whole certificate encloses 1500 samples per box.
- `test_crown_bounds_and_linear_bounds_are_sound[...]`: CROWN's numbers never exceed the true minimum or fall below the true maximum, and its linear functions sandwich the values. CROWN is on average tighter than IBP.
- `test_forward_mode_lie_derivative_matches_autograd` and `test_certificate_lie_derivative_matches_autograd[...]` check forward mode against reverse mode.
- `test_controller_respects_actuator_limits` holds even with weights multiplied by 50, and in interval form.
- `test_spec_verify_state_box_helper` checks the specification helper on positive and negative networks.

`tests/test_z3_verification.py` covers the engines and the certificate:

- `test_z3_finds_genuine_boundary_counterexample`: with $B > 0$ everywhere, a face point is found and validated.
- `test_z3_proves_box_where_barrier_is_negative` returns UNSAT.
- `test_z3_unsafe_set_is_encoded_exactly`: a box inside the obstacle yields a validated unsafe counterexample.
- `test_z3_init_mode` covers P0 in both directions.
- `test_z3_unsat_implies_no_sampled_violation`: every UNSAT is confirmed by 5000 samples.
- `test_full_graph_relaxation_encoding_small_net` exercises engine 2 on a 4-neuron network.
- `test_branch_and_bound_never_certifies_a_violated_network`: an untrained network is not certified, and all reported counterexamples are genuine.
- `test_trivially_empty_barrier_is_not_certified`: P0 catches $B < 0$ everywhere.
- `test_saved_certificate_passes_proof_checker[pendulum|car]`: the pendulum proof is accepted. The car test is skipped, since that result is not a certificate.
- `test_proof_checker_rejects_tampered_certificate`.

18.2 The dashboard (`dashboard/app.py`)

A Streamlit application in five panels:

1. **Status.** Certificate status, number of proven leaves, Z3 leaves, and the proof checker's verdict. A green or amber banner explains the meaning.
2. **Certificate details** (expandable). Weight hashes, properties, the full summary, the per-round plot of counterexamples and unknown boxes, and the checker's per-region report.
3. **Phase portrait.** For the pendulum, the certified set, the unsafe set, X_0, proof leaves and rollouts. For the car, (p_x, p_y) slices at a chosen heading and speed, with rollouts.
4. **Simulation.** Enter any initial state and horizon to see $B(x(t))$ and the states over time, plus a message saying whether the start is inside $\mathcal{C}$ and whether the unsafe set is reached.
5. **Re-verification.** Choose γ, the maximum depth and whether to use Z3, then click *Verify now*. The branch-and-bound verifier runs live, and the first counterexamples are listed if any.

Part VI — Experiments

Chapter 19

Experimental setup

- **Hardware:** a 2-core cloud CPU with about 6 GB of usable RAM. The code targets an 8 GB M2 MacBook Air.
- **Software:** Python 3.11, PyTorch 2.14 (CPU), auto_LiRPA 0.7.2, z3-solver (Python bindings), SciPy 1.17.
- **Numerics:** training in float32; all verification, falsification and simulation in float64.
- **Networks:** two tanh hidden layers of width 32 for both the controller and the barrier.
- **Class-$\mathcal{K}$ function:** $\alpha(B) = B$ ($\gamma = 1$).
- **Configurations:** exactly those in `configs/` (Chapter 16).

Chapter 20

Results

20.1 Inverted pendulum: certified

Synthesis history.

round	counterexamples	proven volume	status	verification time
0	72	97.7%	falsified	67 s (slowed by a parallel job)
1	7	98.9%	falsified	2.9 s
2	0	100%	**certified**	2.7 s

The final proof.

- **Domain:** 279 boxes processed, 140 leaves. 129 were closed by CROWN and 11 by Z3; 49 of the leaves were proven because the whole box lies outside $\mathcal{C}$.
- **Z3:** 57 calls, 11 proofs, 46 spurious candidates.
- **Faces:** 7 leaves in total for the four faces.
- **Initial set:** 1 leaf. The whole of X_0 is proven in a single box.
- **Checker:** accepted in 1.5 s.

Verifier ablation (same networks, `scripts/evaluate.py`):

bounding	status	boxes processed	leaves	time
IBP only	certified	433,573	216,787	2.5 s
IBP + CROWN	certified	329	165	2.5 s
IBP + CROWN + Z3	certified	279	140 (11 by Z3)	2.5 s

Interpretation. The proof shrinks by a factor of about 1,300 from IBP to CROWN, and by a further 15% with Z3. Wall-clock times are similar at this tiny scale because IBP is so cheap per box. In 4-D, box count dominates everything, which is why tighter bounds matter.

γ **sweep:** certified for $\gamma \in \{0.25, 0.5, 1, 2, 4\}$. Values above 1 are implied by Corollary 5.3. Values below 1 happen to hold with these networks too.

Empirical cross-checks:

- 2 M uniform samples: 0 violations of P1 or P2; 200 k face samples and 200 k initial-set samples: 0 violations.
- $\mathcal{C}$ covers 12% of D.
- 50 rollouts from X_0 and 50 from $\mathcal{C}$ over 10 s: none left $\mathcal{C}$, the domain or the safe set. The minimum of B along the rollouts was 0.0003 (from $\mathcal{C}$) and 0.024 (from X_0).

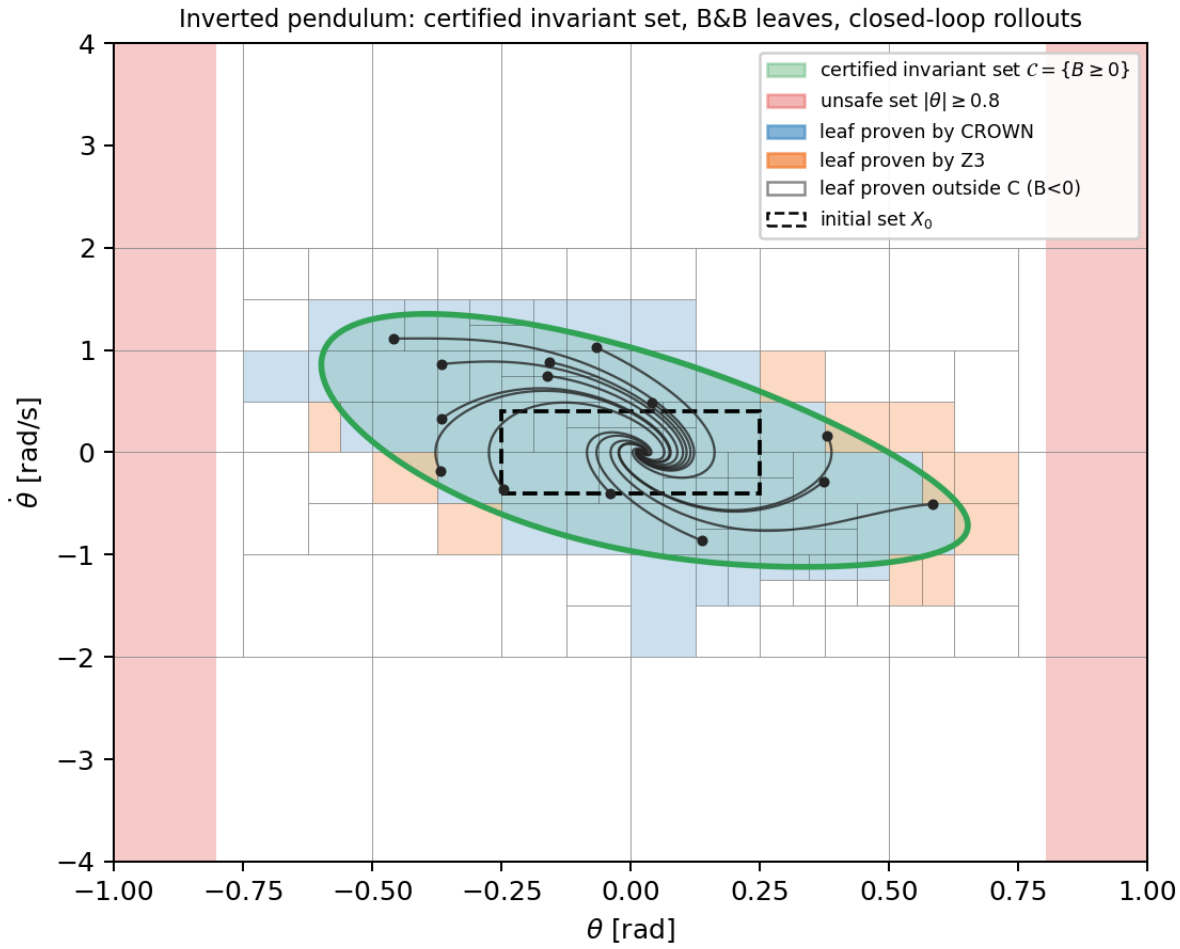


Figure 20.1: Pendulum: certified invariant set (green), proof leaves coloured by engine (blue = CROWN, orange = Z3, grey outline = proven outside $\mathcal{C}$), unsafe set (red), X_0 (dashed) and closed-loop rollouts.

The certified set is a tilted ellipse. States leaning right but already swinging back (upper left and lower right in the $(\theta, \dot{\theta})$ plane) are safe further out than states moving away from upright. $\mathcal{C}$ reaches about $\theta \approx 0.65$ (at negative $\dot{\theta}$), well short of the unsafe set at 0.8, consistent with the torque limit. The controller is smooth, roughly a saturated linear state feedback. Z3 leaves cluster where $\partial\mathcal{C}$ crosses boxes.

20.2 Dubins car: falsified (a partial result)

Final verification (after 40 rounds, `scripts/reverify.py`, 14 minutes):

quantity	value
domain boxes processed	1,073,497
proven leaves	381,104 (378,155 CROWN, 2,949 Z3)
of which outside $\mathcal{C}$	130,220
proven share of vol(D)	**99.44%**
P0 (initial set inside $\mathcal{C}$)	**proven** (22,399 leaves)
validated counterexamples	**524**, which stopped the run early (`max_cex`)
unresolved boxes	277,584 (mostly unexplored because of the early stop)
faces (P3)	not run, because the domain was already falsified
status / checker	**FALSIFIED** / rejected

quantity	value

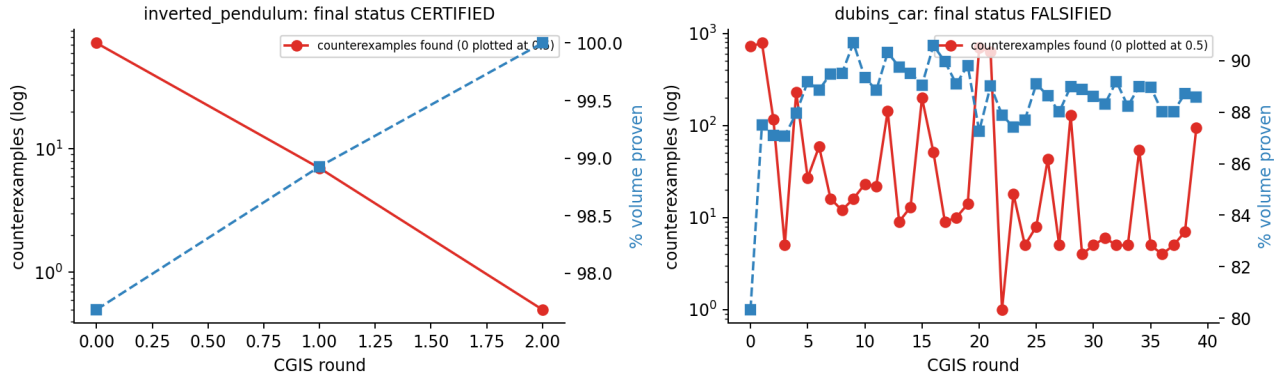


Figure 20.2: Counterexamples per synthesis round and proven volume, for both systems.

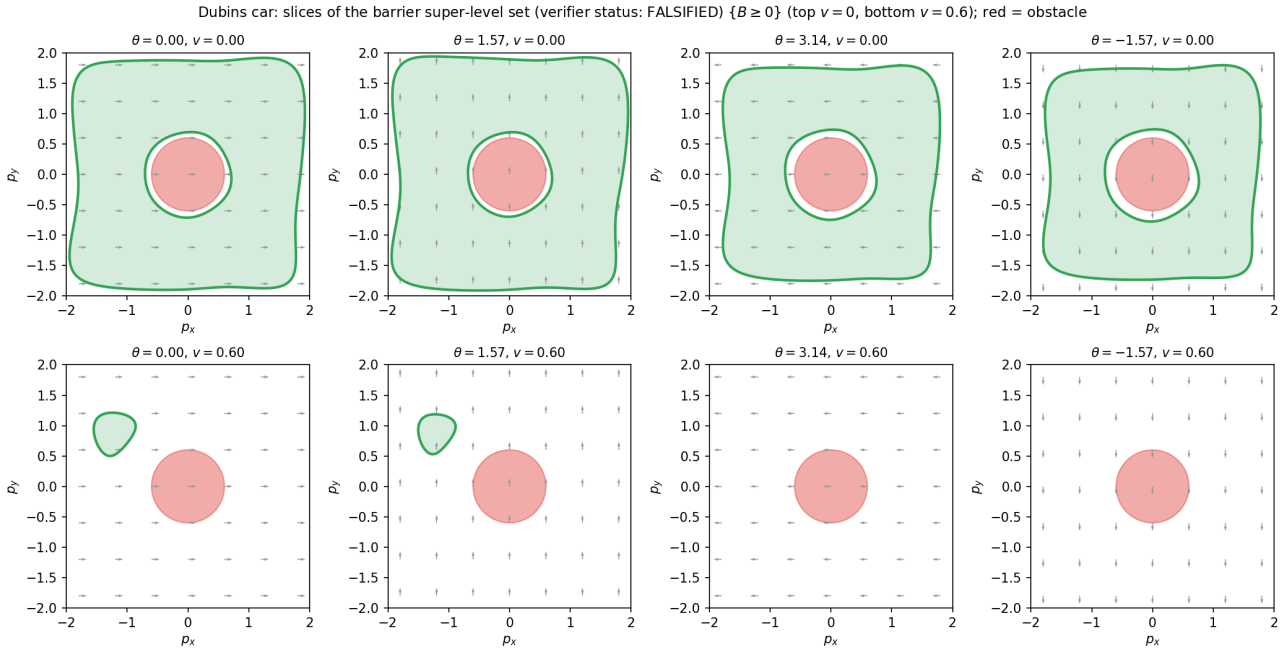


Figure 20.3: Dubins car: slices of $\{B \geq 0\}$ at four headings, for $v = 0$ (top) and $v = 0.6$ (bottom). At higher speed the set shrinks sharply: the policy is conservative.

Where the counterexamples are.

- **Distance to the obstacle centre:** between 0.68 and 0.79 in 80% of cases, just outside the training-inflated obstacle (radius 0.68).
- **Speed:** $|v| < 0.08$.
- **Heading:** pointing almost straight towards or away from the obstacle, $|\cos(\text{relative angle})| \approx 1$.
- **Values:** B between 0 and 0.027; cond between -0.025 and 0.

How visible the failure is empirically.

check	result
2 M uniform samples	23 violations (1.2×10^{-5})
50 rollouts from X_0 and 50 from $\mathcal{C}$	0 left $\mathcal{C}$
50 rollouts starting at counterexamples (3 s)	**41 left** $\mathcal{C}$ (minimum $B = -0.032$); none reached the obstacle

So the counterexamples are real invariance failures. They are not yet collisions, thanks to the inflated safety margin, but the *proof* is broken and the tool correctly refuses to certify.

20.3 Why the car fails: the zero-speed obstruction

Consider a state on $\partial\mathcal{C}$ near the obstacle with $v = 0$. Its position cannot change instantly: $\dot{p}_x = v\cos\theta = 0$ and $\dot{p}_y = v\sin\theta = 0$. So

$$\dot{B} = \partial_\theta B \cdot u_1 + \partial_v B \cdot u_2.$$

Only turning and accelerating can raise B. Now suppose the car is pointing straight at the obstacle. Turning left or right is symmetric, so $\partial_\theta B \approx 0$ there. The condition then requires $\partial_v B \cdot u_2 \geq \tau$: the barrier must reward reversing ($\partial_v B < 0$), and the controller must actually reverse ($u_2 < 0$).

A diagnostic on a mid-run checkpoint (round 18) found $|\partial_v B| \approx 0.3$ at the violating states. The controller used only $|u_2| \leq 0.17$ of the available 2. A stronger reversing command would have satisfied the condition with a large margin.

So a solution exists in principle. The training did not find it: the violating region is thin (sampling rate about 10^{-5}), and the imitation and smoothness terms mildly oppose large accelerations there.

Things that were tried.

1. Smaller margins and higher set-condition weights: fixed P0, P2 and P3 violations, but not P1.
2. A smaller initial set (1.4 instead of 1.6, speed limit 0.25 instead of 0.3): more slack near the walls.
3. Training-only obstacle inflation (0.08): moved the violations *outward* with the boundary, but did not remove them.
4. A lower imitation weight (0.02 → 0.005) and a higher counterexample weight (2 → 5): the counterexample count fell to about 5 per quick round, but never reached zero.

Ideas for finishing it.

- Build velocity-awareness into B, for example $B = \mathrm{NN}(x) - k\,v\cos\varphi$, where φ is the heading relative to the obstacle direction.
- Larger margins together with longer training.
- α/β-CROWN bounds, to reduce the number of boxes.
- Handle $v \approx 0$ near obstacles with a separate analytic braking argument.

Chapter 21

Development log: problems encountered and how they were solved

This chapter records the real engineering history. Each item is a lesson that generalises to other projects.

#	problem observed	root cause	fix	lesson
1	auto_LiRPA crashed with a float/double mismatch	its slice operator allocates buffers with the default dtype	set the default dtype to float64 during CROWN calls	test verification libraries at the precision you intend to use
2	Z3 `check()` hung for minutes and ignored its timeout	exact rationals from floats (denominators up to 2^{52}) blow up in simplex	engine 1: small Z3 problems over CROWN's linear bounds; engine 2 kept for small networks	put each tool where it is strong
3	IBP left 36,000 unknown boxes even at depth 12	dependency problem, and a bang-bang controller with huge local slope	CROWN, plus a controller Lipschitz penalty	verification-friendly training matters as much as the verifier
4	first "certificate" proved nothing: $B < 0$ everywhere	P1–P3 are vacuous for an empty set	added P0 ($X_0 \subseteq \mathcal{C}$) to the verifier; raised the init weight	always certify non-triviality
5	loss plateaued; thin "arms" of $\mathcal{C}$ into infeasible regions	hard mask gives no gradient to shrink $\mathcal{C}$	sigmoid band weight	differentiate through set membership
6	wide band (0.2) stalled training	demanded the condition outside $\mathcal{C}$, which is infeasible	band 0.01	match hyperparameters to the scale of B
7	verification of the car stopped at the root box	a falsified box was dropped without splitting; the root centre lies inside the obstacle	keep splitting large falsified boxes	falsification should not stop exploration
8	the pendulum stopped converging after that change	harvesting 500+ counterexamples per round from tiny boxes flooded the buffer	only split falsified boxes down to `cex_split_depth=4`	too many near-duplicate counterexamples distort training

#	problem observed	root cause	fix	lesson
9	the car run was killed	out of memory: two heavy jobs together exceeded the 6.27 GB container limit	run jobs serially; checkpoint and resume	long jobs must be resumable
10	the checker rejected the final car certificate	the loop trained once more after the last verification	stop right after the last verification	the independent checker catches pipeline bugs
11	rollouts reported "left the domain" while B stayed high	`in_domain` treated the periodic θ as bounded	ignore periodic coordinates	periodic coordinates need care everywhere
12	smarter branching was *worse* (63.9% vs 89.2% volume)	$\lvert A\rvert$·width does not measure relaxation looseness	kept widest-coordinate splitting	measure heuristics before adopting them

Part VII — Perspective

Chapter 22

Related work, limitations and future work

22.1 Related approaches

approach	verifier	networks	guarantee	comparison with this project
Neural Lyapunov Control (Chang et al., 2019)	dReal (δ-complete SMT)	small tanh	δ-sound	dReal handles tanh natively but scales poorly; we use CROWN plus Z3
FOSSIL (Abate et al., 2021)	Z3 / dReal CEGIS	small	sound (δ for dReal)	same CEGIS idea; we add bound propagation and a proof checker
BBV NCBF synthesis (Wang et al., 2023)	branch and bound with linear bounds	ReLU / smooth	sound (floating-point caveats)	closest in spirit; we add the pointwise Z3 step and P0
Survey (Dawson, Gao, Fan, 2023)	various	—	—	broad overview of learned certificates
CBF-QP safety filters (Ames et al.)	none (online QP)	analytic B	depends on a valid B	complementary: filters need a certified B

22.2 Limitations

1. **Floating point.** CROWN has no directed rounding, so the 10^{-8} margin is a heuristic safeguard (Chapter 8).
2. **Model fidelity.** The guarantee is for the ODE model: no sampling, delay, noise or model error.
3. **Scalability.** Box branch and bound is exponential in the state dimension. The 4-D car needs millions of boxes.
4. **Conservatism.** $\mathcal{C}$ is an inner approximation of the viability kernel; it covers 12% of D for the pendulum.
5. **No convergence guarantee** for CGIS (Chapter 13).
6. **Proof checker trust base.** The checker reuses the project's bound code (IBP, CROWN, Z3). It checks consistency and coverage independently of training, but it is not a formally verified kernel.

22.3 Future work

- α/β-CROWN or LP-based relaxations, with optimised slopes.
- Directed rounding or interval-validated CROWN.
- Barrier structures with physics priors (velocity terms), for the car.
- Discrete-time or sampled-data certificates, for real deployment.
- Larger systems (quadrotor, 6–12 states) via learned or partitioned domains.
- GPU batching for bound propagation.

Appendix A — Notation

symbol	meaning
$x \in \mathbb{R}^n$	state
$u \in \mathbb{R}^m$	control input
$f(x)$, $g(x)$	drift, input matrix
π_θ	neural controller with weights θ
B_ϕ	neural barrier with weights ϕ
$\psi(x)$	feature map
F_{cl}	closed-loop vector field $f + g\pi$
$\dot{B}$, $L_F B$	Lie derivative $\nabla B \cdot F_{cl}$
γ	class-$\mathcal{K}$ slope
cond	$\dot{B} + \gamma B$
D	verification domain (box)
$\mathcal{C}$	certified safe set $\{x \in D : B \geq 0\}$
X_0, X_{unsafe}	initial and unsafe sets
m	training margin
τ	verification margin (10^{-8})
δ, T	band width and temperature
$[l, u]$, c, r	box, centre, radius
$\underline{\cdot}$, $\overline{\cdot}$	lower and upper bound
$\underline{A}$, $\underline{b}$	CROWN linear lower bound
u (numerics)	unit roundoff

Appendix B — Glossary

Actuator saturation. Physical input limits, enforced here by tanh. **Adam.** An adaptive gradient optimiser. **auto_LiRPA.** A library for linear relaxation bounds of PyTorch graphs. **Barrier certificate.** A function proving invariance and separation from unsafe states. **Bisection.** Splitting a box in half along one coordinate. **Branch and bound.** Recursive splitting plus bounding until every piece is decided. **Candidate (spurious).** A Z3 model of a relaxation that is not a real violation. **CDCL.** Conflict-driven clause learning (SAT solving). **CEGIS / CGIS.** Counterexample-guided inductive synthesis. **Certificate.** Stored networks plus proof leaves plus metadata. **Class-$\mathcal{K}$.** Continuous, strictly increasing, zero at zero. **Closed loop.** Dynamics with the controller substituted. **Comparison lemma.** Differential inequality ⇒ bound on the solution. **Completeness.** A method is complete if it proves every true statement. **Concretization.** Optimising a linear bound over a box. **Control-affine.** $\dot{x} = f + gu$. **CROWN.** Backward linear bound propagation. **Dependency problem.** IBP's loss of correlations. **DPLL(T).** SAT search combined with a theory solver. **Dual numbers.** $a + b\varepsilon$ with $\varepsilon^2 = 0$ (forward-mode AD). **Enclosure.** Guaranteed lower and upper bounds. **Falsification.** Finding a concrete violating input. **Forward invariance.** A trajectory never leaves the set. **Hinge loss.** $\mathrm{ReLU}(m - q)$. **IBP.** Interval bound propagation. **JVP.** Jacobian-vector product. **Leaf.** A proven box in the branch-and-bound partition. **Lie derivative.** $\nabla B \cdot F$. **Lipschitz.** A bounded rate of change. **McCormick envelope.** Linear hull of a bilinear product. **Mean value theorem.** $f(y) - f(x) = f'(\xi)(y - x)$. **Nagumo's theorem.** Invariance holds iff the flow points inward on the boundary. **Over-approximation.** A superset of the true behaviour. **QF_LRA / QF_NRA.** Linear and polynomial real arithmetic theories. **RK45.** Adaptive Runge–Kutta integrator. **SAT / UNSAT.** Satisfiable / unsatisfiable. **SMT.** Satisfiability modulo theories. **Soundness.** A method is sound if it never claims a false proof. **TCB.** Trusted computing base. **Viability kernel.** The largest set from which safety can be maintained. **Z3.** An SMT solver from Microsoft Research.

Appendix C — Exercises with solutions

C.1 (interval arithmetic). Compute the interval of $x(1-x)$ for $x \in [0,1]$, and compare it with the true range. *Solution.* $1-x \in [0,1]$ and $[0,1]\cdot[0,1] = [0,1]$. The true range is $[0, 0.25]$, with the maximum at $x = 0.5$. The overestimate comes from the dependency problem.

C.2 (Lemma 2.1). Find $\min_{x\in[0,1]\times[-1,2]}(3x_1 - 2x_2)$. *Solution.* Take $x_1 = 0$ (positive coefficient) and $x_2 = 2$ (negative coefficient), giving -4. Check with the formula: $c = (0.5, 0.5)$, $r = (0.5, 1.5)$, $a^\top c = 0.5$, $|a|^\top r = 1.5 + 3 = 4.5$, so the minimum is -4. □

C.3 (McCormick). For $a \in [1,2]$ and $b \in [-1,1]$, give the McCormick bounds at $a = 1.5$, $b = 0$. *Solution.* The lower bounds are $w \geq a_l b + a b_l - a_l b_l = 0 - 1.5 + 1 = -0.5$ and $w \geq a_u b + a b_u - a_u b_u = 0 + 1.5 - 2 = -0.5$. The upper bounds are $w \leq a_u b + a b_l - a_u b_l = 0 - 1.5 + 2 = 0.5$ and $w \leq a_l b + a b_u - a_l b_u = 0 + 1.5 - 1 = 0.5$. So $w \in [-0.5, 0.5]$, which contains the true value 0.

C.4 (Lie derivative). $B(x) = 1 - x_1^2 - x_2^2$ with $\dot{x}_1 = x_2$, $\dot{x}_2 = -x_1 - x_2$. Is $\{B \geq 0\}$ invariant? *Solution.* $\dot{B} = -2x_1x_2 - 2x_2(-x_1 - x_2) = 2x_2^2 \geq 0$ everywhere. So B never decreases, and the unit disc is invariant.

C.5 (why P0). Show that $B \equiv -1$ satisfies P1–P3. *Solution.* P1: $B \geq 0$ never holds, so the implication is vacuously true. P2 and P3: $B = -1 < 0$. So without P0 the empty set would be "certified".

C.6 (γ monotonicity). Why does a proof for $\gamma = 1$ imply one for $\gamma = 3$, but not for $\gamma = 0.5$? *Solution.* On $\mathcal{C}$, $B \geq 0$, so $\dot{B} + 3B \geq \dot{B} + B \geq 0$. For $\gamma = 0.5$ the term $0.5B$ is smaller, and the inequality may fail.

C.7 (hands-on). Set `train.w_init=0` and train the pendulum. Explain what happens. *Expected.* B is driven negative everywhere. The verifier then reports `init_set_certified: false`, so no certificate is issued.

C.8 (hands-on). In `losses`, replace the sigmoid band with `(B >= -tc.lie_band).float()`. Plot $\mathcal{C}$ after training. *Expected.* Thin arms of $\mathcal{C}$ reach into regions where the pendulum cannot be saved, and verification finds counterexamples at their tips.

Appendix D — Frequently asked questions

Is the pendulum controller safe on real hardware? The proof covers the ODE model with the stated parameters. A real system adds discrete-time control, delays and model error. Those need additional margins or a sampled-data analysis.

Why not use ReLU networks, which are easier to verify? $\dot{B}$ needs ∇B, which is discontinuous for ReLU networks. Smooth activations keep the CBF theory standard.

Why not verify on a very fine grid? A grid is still a finite sample. Between grid points, a function can do anything unless you also bound its variation, which is exactly what interval and CROWN bounds do.

What does UNKNOWN mean in practice? The budget ran out. Increase `max_boxes` or `max_depth`, or retrain with larger margins.

How do I add a new system? Subclass `ControlAffineSystem`, implement `xdot`, `features`, `features_jvp` and `unsafe_pieces` (optionally `init_constraints` and `nominal_control`) using `ops.*`, register it in `make_system`, and add `configs/system/<name>.yaml`.

How much does each component cost? Per box: IBP about a microsecond, CROWN about 0.5 ms (batched), Z3 a few ms. Training a round: seconds to minutes.

Appendix E — References

1. A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, P. Tabuada. *Control barrier functions: theory and applications.* ECC, 2019.
2. S. Prajna, A. Jadbabaie. *Safety verification of hybrid systems using barrier certificates.* HSCC, 2004.
3. M. Nagumo. *Über die Lage der Integralkurven gewöhnlicher Differentialgleichungen.* Proc. Phys.-Math. Soc. Japan, 1942.
4. H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, L. Daniel. *Efficient neural network robustness certification with general activation functions.* NeurIPS, 2018.
5. K. Xu, Z. Shi, H. Zhang, et al. *Automatic perturbation analysis for scalable certified robustness and beyond.* NeurIPS, 2020.
6. S. Gowal et al. *On the effectiveness of interval bound propagation for training verifiably robust models.* arXiv:1810.12715, 2018.
7. L. de Moura, N. Bjørner. *Z3: an efficient SMT solver.* TACAS, 2008.
8. G. P. McCormick. *Computability of global solutions to factorable nonconvex programs: Part I.* Mathematical Programming, 1976.
9. A. Solar-Lezama, L. Tancau, R. Bodík, S. Seshia, V. Saraswat. *Combinatorial sketching for finite programs.* ASPLOS, 2006.
10. G. Katz, C. Barrett, D. Dill, K. Julian, M. Kochenderfer. *Reluplex: an efficient SMT solver for verifying deep neural networks.* CAV, 2017.
11. Y.-C. Chang, N. Roohi, S. Gao. *Neural Lyapunov control.* NeurIPS, 2019.
12. A. Abate, D. Ahmed, A. Edwards, M. Giacobbe, A. Peruffo. *FOSSIL.* HSCC, 2021.
13. C. Dawson, S. Gao, C. Fan. *Safe control with learned certificates: a survey.* IEEE Transactions on Robotics, 2023.
14. X. Wang, L. Knoedler, F. B. Mathiesen, J. Alonso-Mora. *Simultaneous synthesis and verification of neural control barrier functions through branch-and-bound verification.* arXiv:2311.10438, 2023.
15. N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM, 2002.
16. S. Gao, S. Kong, E. M. Clarke. *dReal: an SMT solver for nonlinear theories over the reals.* CADE, 2013.